njection attack detection, discrete event logs, semi-Markov processes, feasibility mask, conformal prediction, sequential testing, cyber-physical production systems

Reference-Model Hard Constraints and Semi-Markov Timing Channels for Detecting Injection Attacks on Industrial Discrete-Event Streams

QI YU, School of Cyber Science and Engineering, Southeast University, PR China

WEI LI*, School of Computer Science and Engineering, Southeast University, PR China

In cyber-physical production systems, field devices report their operation as discrete activity events that carry a lifecycle (assignment, start, completion). By injecting or replaying such events on the uplink, an attacker can decouple the log from the physical process. Detectors that threshold only the predicted probability of labels have a structural blind spot here: under a zero-false-alarm constraint they can flag only transitions that the process reference model already forbids, and against forgeries that leave labels intact and merely compress sojourn times, their net detection is zero. We build a detector with three parallel paths. A hard-constraint layer vetoes infeasible events outright using a feasibility mask derived from the reference model; a semi-Markov timing channel produces residuals from grouped log-normal sojourn times; and a structural channel measures whether an activity transition within a process instance is rare. The two statistical channels are calibrated by conformal prediction and accumulate evidence message by message through cumulative-sum (CUSUM) tests, with the false-alarm budget split evenly across the three paths. On the theoretical side, we characterize the blind spot of the label-only detector family in a proposition and derive the critical relative advance $\rho^*(\alpha, \sigma)$, the fraction by which a sojourn must be shortened for a single forgery to be detected with probability one half. We synthesize six attack classes under a reproducible protocol on a real production log from the Fischertechnik Trier smart factory. With a false-alarm budget $\alpha = 0.01$, a delay budget of 10 messages, and the chance-alarm floor subtracted, our net detection is 0.95 for physically infeasible injection and 0.87 for gradual drift, where the label-only ablation scores 0. For early-completion replay at a 20% injection rate it is 0.18, higher than the timed-automaton baseline (significant in a paired test) and not significantly different from the baseline that combines timed automata with a Bayesian network. On state mimicry and message suppression, our method falls below the label-only ablation. The conclusions rest on a single public log and synthetic injections; when thresholds are instead calibrated on the calibration fold, every method exceeds its nominal false-alarm rate.

CCS Concepts: • **Security and privacy** → **Intrusion detection systems**; • **Computer systems organization** → *Embedded and cyber-physical systems.*

ACM Reference Format:

Qi Yu and Wei Li. 2026. Reference-Model Hard Constraints and Semi-Markov Timing Channels for Detecting Injection Attacks on Industrial Discrete-Event Streams. 1, 1 (October 2026), 30 pages. https://doi.org/XXXXXXX.XXXXXXX

*corresponding author

Authors' Contact Information: Qi Yu, 230240012@seu.edu.cn, School of Cyber Science and Engineering, Southeast University, Nanjing, Jiangsu, PR China; Wei Li, xchlw@seu.edu.cn, School of Computer Science and Engineering, Southeast University, Nanjing, Jiangsu, PR China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

1 Introduction

1.1 Background and Motivation

In a cyber-physical production system (CPPS), field devices report their operation as discrete activity events: each event carries a device, an operation (or state label), a timestamp, and a process instance [25, 30]. For example, after the scheduling system assigns a transport device to "move to a given station", the log records the assignment, the start of execution, and the completion in turn, so every activity has a lifecycle from assignment to completion. An attacker need not rewrite a continuous control law or compromise device firmware: **injecting or replaying these events** on the uplink suffices to decouple the trajectory recorded in the log from the physical process. Detection research on industrial cyber-physical systems concentrates on physical process variables, perimeter protection, and network traffic [12]; lifecycle activity events are rarely treated as an injection target in their own right. We detect whether these events have been injected or replayed; how the scheduler dispatches work on the basis of them is outside the scope of the evaluation.

A typical attack replays a "completed" event before the activity has actually finished. The record is entirely legitimate at the label level, yet it rewrites the sojourn time and the context of the events that follow. A detector that only compares predicted and observed labels cannot recognize such an injection: its decision depends only on the (predecessor, label) pair and is identical whether the record is benign or forged (Proposition 5.1).

The same event stream, however, carries three kinds of structure that a detector can exploit, each matched to a different type of injection. The first is *feasibility*: the process specifies which device may execute which operation, and from which predecessor operation that operation may be reached. These constraints are given by the reference process model and cannot be learned from operational logs, because a log records only the combinations that have occurred and cannot distinguish "never observed" from "not allowed". The second is *duration*: activities consume physical time, and an event whose label sequence is legal but whose timing is wrong is indistinguishable from a normal event under a label-only model; only the sojourn time can expose it. The third is *workflow structure*: the transition frequencies between activities within a process instance, which change when activities are inserted or deleted. In addition, the causal pairing between a completion event and an assignment recorded in the log can serve as an extra check in the hard-constraint layer.

1.2 Gap in Existing Work

Existing detectors do not exploit these structures, and the reason lies in their methodological premises. To keep the state space tractable, timed automata for production lines [21] decompose the system into components that are modelled independently, so cross-component feasibility constraints cannot be expressed. TABOR [18] combines timed automata with Bayesian networks, but confines dependencies to a single stage and does not use assignment (command) data. The deterministic counting timed automaton DCTA [4] constrains communication-protocol sessions in smart-grid SCADA and is unrelated to the production process. Work on resilience against injection and replay at the continuous control layer [9, 24] deals with continuous state estimates such as position and velocity rather than discrete activity events. Section 2 gives a detailed comparison.

1.3 Our Approach

We exploit the three kinds of structure with three parallel paths. A hard-constraint layer vetoes infeasible events outright with a feasibility mask derived from the reference model, and additionally checks assignment–completion

pairing. A timing channel groups activities by (device, operation) and produces standardized residuals from log-normal sojourn times. A structural channel uses the workflow transition probabilities within a process instance to measure how rare an activity is. The two statistical channels are calibrated by conformal prediction and accumulate evidence message by message through CUSUM tests.

Timing models are not new in themselves: timed automata have been applied to timing anomaly detection on production lines and in SCADA [17, 21], and modelling sojourn times with hidden semi-Markov models is established practice [33]. Our work differs from these in two respects. First, the clock guard of a timed automaton is a binary interval decision, so a small early report that stays inside the interval is simply accepted; a distributional sojourn model instead turns such same-direction deviations into evidence that can be accumulated. Second, data-driven models flag activity combinations absent from the training data as anomalous and thus mistake new product variants for attacks, whereas reference-model feasibility rejects only combinations that the process does not allow.

1.4 Main Results

The experiments use a false-alarm budget $\alpha = 0.01$ and a delay budget of 10 messages (an injection counts as detected if an alarm is raised within 10 messages after it), and the reported net detection rates have the chance-alarm floor—the detections produced by random false alarms within the window—subtracted. The six attack classes are naive replay (A1), physically infeasible injection (A2), early-completion replay (A3), state mimicry (A4), gradual drift (A5), and message suppression (A6), defined in Section 3.4. The main results are as follows. A2 is covered by the device capability sets prescribed by the reference model, with a net detection of 0.95. On A5 our method reaches 0.87, while the label-only ablation, which drops the time dimension, scores 0.00. On A3 (at a 20% injection rate) our method reaches 0.18±0.01, higher than the 0.10 of BUTLA (paired test, Holm $p = 0.031$); its difference from the TABOR-style baseline is not significant after correction ($p = 0.096$), and the advantage disappears at injection rates of 10% and below. On A4/A6 our method falls below the label-only ablation (see Section 7.2 for the reasons). When thresholds are calibrated on the calibration fold instead, every method exceeds its nominal false-alarm rate (ours: 0.030). The single-message power curve predicted by the per-operation mixture model of the critical relative advance agrees with the measurements, with a mean absolute deviation of about 0.03. Details are given in Section 6.

1.5 Contributions

Our contributions are as follows.

(1) **We formalize injection detection on discrete activity events with a lifecycle.** The detection target is the lifecycle activity event (assignment, start, completion)—neither the continuous control layer nor a single process variable nor a network packet session. We give the system model, the threat model, and reproducible injection rules for six attack classes (Table 2); detection focuses on injections that are infeasible under the reference model and on injections whose labels are legal but whose sojourn times are misaligned.

(2) **We build a hard-constraint layer from the reference-model feasibility mask.** The device-capability check covers every message and is never violated on the cleaned normal data (0.003 on the raw log with execution errors); none of the 20/508 benign new combinations in the temporal test fold is rejected. This constraint cannot be learned from operational logs.

(3) **We detect label-legal duration forgeries with distributional sojourn times and sequential accumulation.** Sojourn times are modelled per (device, operation) group; the timing and structural channels are calibrated by conformal prediction and accumulated by CUSUM tests. Unlike an interval guard, this accumulates same-direction early reports that stay inside the normal interval (see Section 5.2 for the conditions).

(4) **We state the impossibility result for the label-only detector family as a numbered proposition and derive the critical relative advance $\rho^*(\alpha, \sigma)$.** For label-consistent injections, a detector that thresholds only predicted probabilities adds nothing beyond the feasibility mask. ρ^* is the point at which the single-message detection probability reaches one half; it depends only on the timing variability and the false-alarm budget, and it instantiates the fundamental limits of stealthy attacks [19] for discrete events without a system matrix.

Sections 2–7 present related work, the problem statement, the detection method, the theoretical analysis, the experiments, and the conclusions, in that order.

2 Related Work

The work closest to ours models industrial processes as timed automata or semi-Markov processes and detects anomalies through timing constraints.

2.1 Timed Automata and Semi-Markov Models

Maier et al. [21] learn timed automata on production lines with the Bottom-Up Timing Learning Algorithm (BUTLA). They first derive the component topology and parallel structure from PROFINET adjacency, and then learn a separate behaviour model for each component; at detection time a single automaton checks whether a symbol is legal, whether its duration falls inside the clock guard, and whether the transition is too rare. The method assumes sequential execution within a component, and components are split to avoid an explosion of the joint state space; consequently it neither checks cross-component consistency nor involves assignment–completion pairing. Its clock guard is a binary interval decision, so a small early report inside the guard is accepted. Maier et al. also note that the tolerance of their probabilistic check must be set manually and that, because a perfectly correct model cannot be learned, normal behaviour is sometimes misclassified as erroneous—avoiding this would require infinitely many test samples. In our method the user specifies the false-alarm budget α instead, and thresholds are calibrated on benign data (Section 6.6 reports the deviation under temporal deployment).

Lin et al. [17] apply timing-based anomaly detection to SCADA networks; they likewise attach time windows to discrete events, but their object is message timing rather than process feasibility. Tan and Xi [33] replace the hidden Markov model with a hidden semi-Markov model precisely because the exponential sojourn assumption is unrealistic; the model is now an established tool in anomaly detection. Sojourn-time modelling as such is therefore not a contribution of this paper.

TABOR [18] combines timed automata with Bayesian networks for one-class classification on the SWaT water-treatment testbed, with an emphasis on interpretability and anomaly localization. Its combination resembles our "timing plus dependency" design, but each Bayesian-network submodel is confined to a single stage: the original paper states explicitly that it learns the dependence of sensors on actuators within the same stage. The work also declares that it focuses on the physical process and does not use network traffic, so it contains no assignment–completion pairing. The official SWaT attack scenarios already include cross-stage attacks [10, 18], yet TABOR has no cross-stage component and can cover such attacks only through the union of independent per-stage detections.

In summary, we differ from these works in two respects. The first is duration: we obtain distributional sojourn times from a semi-Markov kernel rather than binary clock-guard intervals, so same-direction early reports inside the normal interval can still be accumulated sequentially. The second is feasibility: Maier et al. give up cross-component coupling to keep learning tractable and TABOR confines dependencies to stages, whereas we implement the feasibility mask derived from the reference model as a hard constraint.

2.2 Packet-Level State Machines

Goldenberg and Wool [11] characterize Modbus/TCP sessions with deterministic finite automata. DCTA [4] goes further and models IEC-104 communication in smart-grid SCADA as a deterministic counting timed automaton, dividing anomalies into single-packet, packet-sequence, and timing anomalies. This division resembles our hard-constraint layer, structural channel, and timing channel on the surface, but the two operate at different levels: DCTA constrains packet attributes, event timing, and protocol counters, its modelling unit is the session between a master station and a remote terminal unit, and it involves neither the production process nor activity semantics.

2.3 Continuous Control and Cyber-Physical Security

Attack defences for automated guided vehicles and formation control deal with continuous state estimates such as position and velocity, for example robust Kalman localization against false data injection and denial of service [9]. Cyber-physical security research since Mo and Sinopoli [24] studies replay, watermarking, and detection delay on linear systems. Neither line of work targets discrete activity events or uses the feasibility mask given by a process reference model.

2.4 Sequential Testing, Conformal Prediction, and Stealthy-Attack Limits

CUSUM and quickest change detection are widely used to detect false data injection in power grids [16], and the asymptotic optimality of their detection delay goes back to Lorden and Lai [14, 20]. Active watermarking and detection-delay bounds now form a complete theoretical toolkit [26]. The fundamental limits of stealthy attacks on continuous linear systems have been established systematically [19].

Conformal prediction provides distribution-free false-alarm control [2, 31], but the guarantee relies on exchangeability, and coverage degrades under temporal drift [3]. Conformal martingales can test online whether exchangeability is violated [36] and have been applied to change-point detection [35]; we do not use them. Conformal anomaly detection for industrial cyber-physical systems has been studied specifically, including sliding calibration with temporal quantile adjustment [37] and conditional false-alarm control in thermal-power maintenance [13]. We use all of these tools but do not claim them as contributions; the critical relative advance $\rho^*(\alpha, \sigma)$ introduced later is merely an instance of the fundamental limits of stealthy attacks in the discrete-event setting, distinguished by depending only on timing variability rather than on a system matrix.

2.5 Conformance Checking in Process Mining

Process mining uses event logs for conformance checking and performance analysis against a reference model [1, 34], and the time perspective can enter the alignment cost [29]. This line of work usually computes fitness offline over complete traces; it has no attacker model and offers no user-specified false-alarm budget α. If conformance were used directly as a detector, with the rule "alarm on any feasibility violation", its decisions would coincide with those of our hard-constraint layer: it would cover physically infeasible injection but detect nothing of duration attacks that leave

Table 1. Related work by level. Existing work neither rejects physically infeasible injections through reference-model feasibility nor accumulates duration deviations that stay inside the guard.

Level	Representative	Duration	Feasibility	Assignment pairing
(i) Decomposed production line	Maier/BUTLA [21]	Clock guard	No cross-component check	No
(ii) Single physical process	TABOR [18]	Clock guard	Within-stage dependency only	No
(iii) Packet session	DCTA [4], Modbus DFA [11]	Event timing, counters	No process constraint	No
(iv) Online event injection (ours)	Semi-Markov + reference model	Distributional sojourn	Feasibility mask in hard-constraint layer	Yes

labels unchanged. The hard-constraint column of Table 4 gives the single-message results of this conformance proxy. We use the same kind of reference model (the BPMN released with the dataset), but the problem we address is online, event-by-event injection detection.

Sequence anomaly detection on system logs usually trains a next-log-key predictor and flags an anomaly when the actual key is not among the top few predicted candidates [8, 15, 23]; anomaly detection on event logs in process mining relies mainly on autoencoders [27] or next-event prediction [28]. A next-event predictor without time features decides only on (history, label) and therefore belongs to the label-only detector family: replacing the predecessor i in Proposition 5.1 by a finite history leaves the proof unchanged. Learned models with time features are not bound by the proposition; we do not include them as baselines (Section 7.2).

2.6 Positioning

Table 1 compares related work at four levels. Our work sits at level (iv): online event-injection detection with a reference model. The key difference from level (i) is that its component-decomposition assumption rules out cross-component feasibility; the key difference from level (ii) is that its dependencies are confined to stages. Component decomposition and stage confinement prevent physically infeasible injection (A2) from being rejected by cross-component feasibility, and the binary decision of a clock guard prevents label-legal duration attacks (A3/A5) from being detected by accumulation.

3 Problem Statement and Threat Model

3.1 System Model: The Assignment–Event Loop

A cyber-physical production system consists of a supervisory execution system $\mathcal{C}$ and a set of heterogeneous field devices $\mathcal{D} = \{d_1, \dots, d_M\}$ connected by an industrial network. Devices receive downlink assignments and report discrete activity events. Our object of study is the integrity of this event stream, not continuous control laws or autonomous coordination among devices.

Each event is written $m_t = (d_t, o_t, t)$, where $d_t \in \mathcal{D}$ is the device, o_t the operation or state label, and t the timestamp; the event also carries a process-instance identifier (hereafter the *case*), which we omit when no confusion arises. Each event is transmitted over the uplink as one message, and we use "event" and "message" interchangeably. The

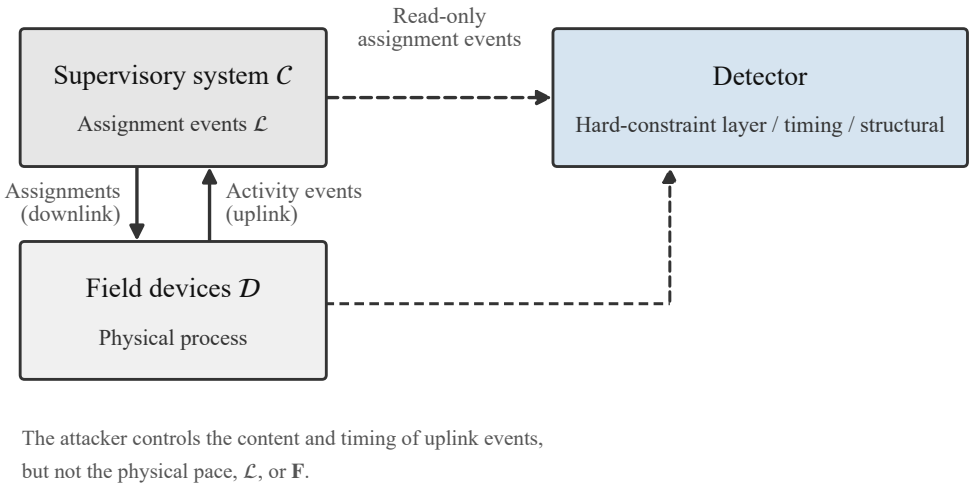


Fig. 1. The assignment/event loop. The detector reads only the assignment events in the same log and the reference-model feasibility mask, and decides online on each uplink event.

binary reachability check maintains predecessors per (device, case), and the structural channel maintains predecessors per case; the feasibility mask constrains reachability only within a single workflow instance and never across case boundaries.

Two reporting semantics must be handled separately. Under *event-driven* reporting, an event is sent when the state changes, and the sojourn time is directly observable. Under *periodic polling*, the state is sampled at a fixed period and the same state is reported repeatedly, so only the interval containing the sojourn time is observed (interval censoring). The production log in our main experiments is event-driven; the method keeps a branch for polling, but it is not evaluated. Figure 1 shows the loop structure and where the three detection paths sit.

3.2 Assignment Pairing and the Feasibility Mask

The same event log also records the downlink assignments, whose set we denote by $\mathcal{L}$. A completion event should be the response to some assignment: if a completion event has no preceding assignment on the corresponding (device, case), it is a causal gap that can be decided in a single step. Once a smart factory integrates process management bidirectionally with the shop-floor event stream, this pairing can be read directly from the log [30]. Assignment–completion pairing is a causal check, not a duration test: the queueing delay between assignment and start is driven mainly by device contention and should not enter the timing channel. The check is retained as an implementation detail of the hard-constraint layer and is not evaluated separately (Section 4.2).

Feasibility is derived from the reference process model released with the production line (BPMN in our case) and denoted by the mask F, which contains two kinds of hard constraint. A *unary capability set* specifies whether device d may execute operation o; it does not depend on the predecessor and therefore covers every message. *Binary reachability* specifies whether, on the same case and the same device, operation o may follow the confirmed predecessor operation o'. Neither can be learned from operational logs: a log records only the combinations that have occurred and cannot

distinguish "never observed" from "not allowed". The same reference model also yields cross-device constraints such as material-token conservation (as a workpiece moves between stations, every pick should correspond to an earlier place), but without workpiece-level identities such constraints suffer residual benign violations, and we do not include them in the detector (see Section 7.2).

3.3 Modelling Assumptions and Threat Model

We make two modelling assumptions. First, the detector reads only uplink events and the assignments in the same log; how the supervisory system dispatches work on their basis is not evaluated. Second, the reference model is consistent with the cleaned normal log, on which the hard-constraint layer has zero violations; the raw log with execution errors does not satisfy this assumption (Table 10).

Attacker capabilities follow the Dolev–Yao model [7]: the attacker acts on the uplink event channel, can eavesdrop on, replay, inject, drop, or delay messages, and may know the state-machine structure and even the transition frequencies. The attacker may know the structure and parameters of the detector (including the σ of each group) but cannot observe its online state (such as the CUSUM statistics and reset times). The attacker cannot compromise the supervisory system or device firmware, cannot change the pace of the physical process, and cannot rewrite assignment events already written to the log or the reference model. The attacker's goal is to decouple the event record from the physical process. Coordinated forgery across multiple devices, which requires aligning several event streams and assignments at once, is outside our scope.

3.4 Attack Classes

Table 2 lists the six injection classes and their reproducible rules. A1, naive replay, is the basic replay reference. A2, physically infeasible injection, rewrites the operation label so that the event violates the reference model. A3, early-completion replay, and A5, gradual drift, leave labels unchanged and only compress sojourn times: the former samples activities sporadically, the latter acts on a run of consecutive activities. A4, state mimicry, assumes the attacker knows the transition model and inserts the most likely next step. A6, message suppression, deletes an entire activity that has not yet started. The primary targets are A2 and A3/A5; A1 serves as the replay reference, and A4/A6 delimit the boundary of the detection capability.

3.5 Detection Problem

Problem 1 (Discrete-event injection detection). *Given a historical log of normal operation only, a reference process model, and a user-specified false-alarm budget α, raise alarms on the online event stream message by message, with the goal that the per-message alarm rate on a purely benign stream does not exceed α. Under exchangeability this goal is guaranteed by conformal calibration; the measured deviation under temporal deployment is reported in Section 6.6. The primary targets are A2 and A3/A5; results on A1/A4/A6 are reported as well. The detector must not use attack labels to tune thresholds. The security goal is* to detect a forgery, or to force its relative advance to stay small*: once the relative advance of a single forgery exceeds $\rho^*(\alpha, \sigma)$, the single-message test detects it with probability greater than one half (Section 5.3).*

We use three evaluation metrics: the net detection rate within the delay budget at a common α (detection rate minus the chance-alarm floor, see Section 6.1); the detection delay in messages (box plots, covering only injections detected

Table 2. Reproducible rules for the six injection classes. Default injection rate 20%, default $\rho = 0.30$ for A3/A5, three injection seeds; the case is never altered. A4 requires a structural model, without which it degenerates into repeating the current operation and underestimates the attacker. By default the evaluation does not shift the subsequent events of the same instance forward.

ID	Class	Rule
A1	Naive replay	Resend a historical activity of the same device after the current completion time; the historical duration is kept and the label is duplicated.
A2	Physically infeasible injection	Rewrite the operation name in place to a label outside the device's capability set (or to another operation if none exists); subsequent benign events are checked for binary reachability with the forged operation as predecessor (hereafter the cascaded check).
A3	Early-completion replay	Move the completion time earlier in place so that the sojourn time becomes $(1-\rho)\tau$; by default, subsequent events of the same instance are not shifted.
A4	State mimicry	With the model known, insert the most probable next step after the current completion; the device is one that has executed this operation in the log, and the duration is the median of that group.
A5	Gradual drift	Take a run of consecutive activities in time order and compress each sojourn by the same ρ (in contrast to the sporadic sampling of A3).
A6	Message suppression	Delete an entire activity that has not yet started; the label is placed on the next event of the same instance (the observable consequence appears downstream).

within the budget); and the sequential false-alarm rate and alarm intervals on a purely benign stream (ARL_0 is not estimated from them when alarms are too few). The computational constraint is $O(1)$ per message.

4 Detection Method

4.1 Overall Architecture

The production configuration consists of three parallel paths—the hard-constraint layer, the structural channel, and the timing channel—and an alarm is raised when any of them fires. The offline phase uses only normal logs and the reference model: F is derived from the BPMN; transition probabilities and sojourn times are estimated per device and operation; the two statistical channels are calibrated by conformal prediction on a held-out calibration fold; and a CUSUM threshold h_i is calibrated for each path on the benign stream so that the path's average run length to false alarm is $\mathrm{ARL}_{0,i} = 1/\alpha_i$, where α_i is the budget assigned to the path (see the next paragraph). The online phase performs a constant number of table lookups and logarithms per message, as shown in Figure 2 and Algorithm 1.

The false-alarm budget is split evenly across the three paths, $\alpha_i = \alpha/3$: the single-message threshold is $\alpha/3$, and the CUSUM threshold corresponds to $\mathrm{ARL}_{0,i} = 3/\alpha$ (300 at $\alpha = 0.01$). Once the three paths are combined by logical OR, the overall single-message false-alarm rate is at most α (the Bonferroni bound), and the overall sequential alarm rate is at most the sum over the paths, so the overall ARL_0 is roughly no less than $1/\alpha$. The hard-constraint layer never fires on normal data and in practice consumes none of its share, so both bounds are conservative. The price of parallel decisions is the split: every method divides the same budget evenly across its sub-detectors. The label-only ablation has a single path and keeps all of α, BUTLA and HSMM have two paths each, and the TABOR-style baseline has three;

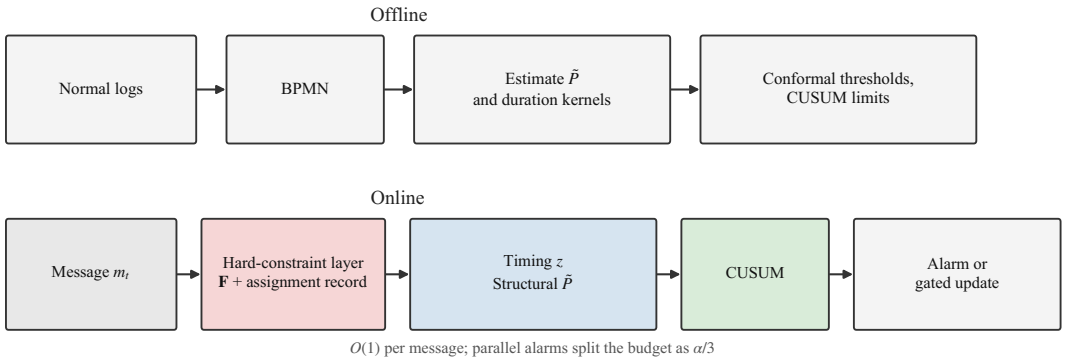


Fig. 2. Offline fitting and online three-path parallel decisions. The hard-constraint layer precedes the statistical channels so that infeasible samples do not contaminate calibration.

Algorithm 1 Online detection of discrete-event injection

Require: event $m = (d, o, t)$; assignments $\mathcal{L}$; mask F; kernels $(\tilde{P}, \mu, \sigma)$; conformal calibrators; thresholds h
Ensure: alarm or accept

if the unary capability, binary reachability, or assignment–completion check fails **then**
    return alarm (hard-constraint reason)
end if
compute the timing residual z and the structural score $\tilde{P}_{o' \to o}$
convert each score to a conformal p-value p with the frozen conformal calibrator
update $S \leftarrow \max(0,\ S - \log p - c)$ for each of the two channels
if $S \geq h$ on either channel **then**
    reset that channel; **return** alarm
end if
if no alarm within the quarantine window **then**
    gated update of (μ, σ) and anchor the state
end if
return accept

on an attack where only one path carries signal, each of our paths has only one third of the budget available to the label-only ablation. Section 6.3 reports the measured effect of this price.

The hard-constraint layer must come first: its decisions produce no false alarms and are interpretable, and placing it first keeps infeasible samples out of the subsequent calibration. The structural and timing channels run side by side rather than in series, so that both "wrong label" and "right label at the wrong time" are covered. The sequential test comes last, so both statistical channels must output continuous scores rather than Boolean decisions.

Figure 3 traces two real cases message by message. A2 is vetoed immediately by the unary capability set with a readable alarm reason; the labels of A3 are entirely legal, and its negative residuals, which lie inside the interval guard, cross the threshold after CUSUM accumulation.

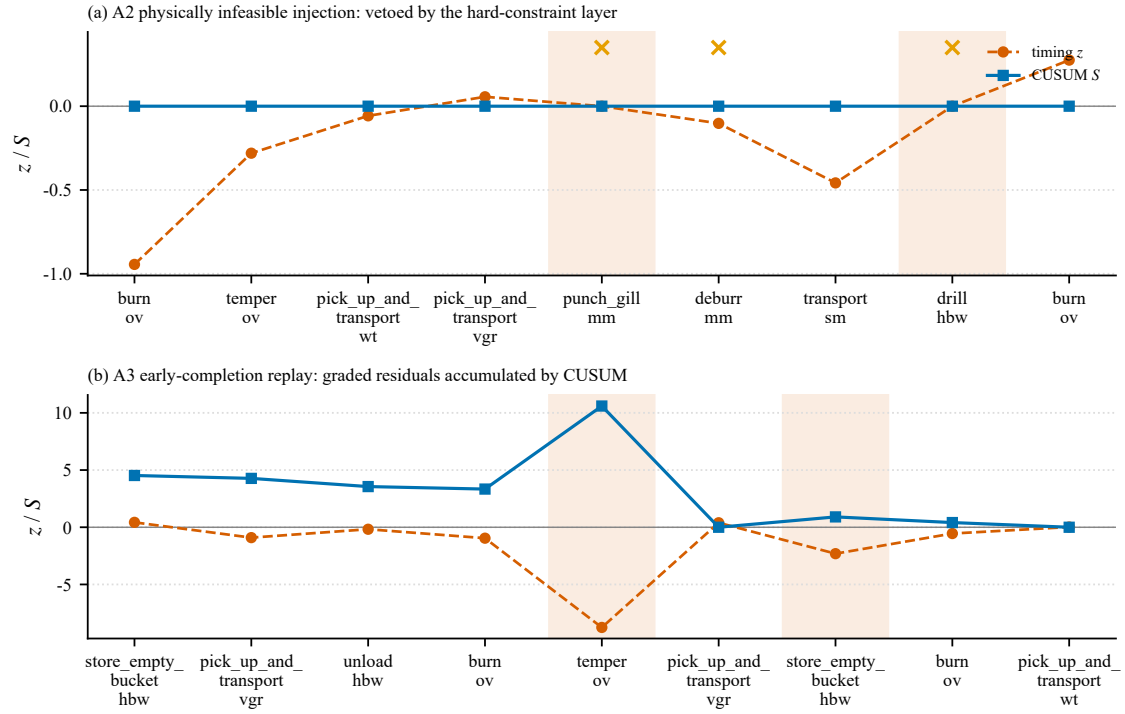


Fig. 3. Message-by-message trace of the hard-constraint veto on A2 and of timing-evidence accumulation on A3. Shaded columns are injected messages; crosses mark hard-constraint alarms.

4.2 Hard-Constraint Layer

For a message (d, o, t), the hard-constraint layer performs three checks in order; any failure raises an alarm immediately, together with its reason.

(1) Unary: whether (d, o) belongs to the capability set of the reference model.

(2) Binary: if the same (d, case) has a confirmed predecessor o', whether F allows $(o' \to o)$.

(3) Causal: whether the completion event has a preceding assignment event.

The mask is derived automatically from the BPMN released with the dataset: unary capability sets come from the device types (or resource roles) bound to the nodes, and binary reachability is obtained by unfolding the sequence flows and gateways within each workflow, so no transition table has to be written by hand. The unary check covers every message and is never violated on normal data. The value of the reference model is most evident under a temporal split: 20/508 (3.9%) of the benign activities in the test fold are (device, operation) combinations absent from the training fold, far more than $\alpha = 0.01$, so a rule of "unseen in training means anomalous" would generate a large number of false alarms, whereas the reference model rejects none of them. The third, causal, check is an implementation detail: none of the six injection classes breaks assignment–completion pairing, so we neither evaluate it separately nor count it as a contribution.

4.3 Structural Channel

The structural channel judges whether the current activity is common in the case-level workflow sequence. The chain is kept per case rather than per (device, case): the devices in this log are service endpoints that are invoked, so most per-device event chains have length 1 and their transition probabilities carry little discriminative information. The channel state is the operation name rather than the (device, operation) pair: "the wrong device executing the right operation" is already covered by the unary capability set, and the statistical channel should not additionally assign very small probabilities to unseen combinations, because under a temporal split benign data itself produces new combinations (Section 6.4 reports the measured cost of using (device, operation) instead). Transition probabilities are the posterior predictive $\tilde{P}$ of a symmetric Dirichlet prior with concentration 0.5 (Jeffreys-type), which avoids spurious zeros caused by small samples while preserving the true zeros enforced by F.

The score fed to the conformal calibrator is the predicted probability $\tilde{P}_{o' \to o}$ itself, for reasons given in Section 4.5. The first activity of a case has no predecessor, so the structural channel abstains and its score is treated as least anomalous. The channel is relatively sensitive to attacks that change the number of events, such as A4/A6.

4.4 Semi-Markov Timing Channel

The semi-Markov kernel is written $Q_{o'o}(\tau) = P_{o'o} F_{o'o}(\tau)$. Sojourn times are grouped by (device, operation) and modelled as $\log \tau \sim \mathcal{N}(\mu, \sigma^2)$. The difference from a timed automaton is that $F_{o'o}$ is a probability distribution rather than an interval guard (Section 5.2). The location parameter μ is decomposed additively by travel route (source and destination storage positions): routes with at least 8 observations in a group get their own effect, and the rest are pooled into the group's common intercept; unseen routes fall back to the planned duration plus the group's systematic offset. Groups that are absent from the training fold or have fewer than 2 observations, as well as the abstaining groups described below, contribute no timing evidence, and their scores are treated as least anomalous; 56 of the 508 activities in the test fold (11.0%) fall into this category.

Four kinds of observation correspond to four branches.

- **State transition.** The standardized residual $z = (\log \tau - \mu)/\sigma$, tested on the left side only (a sojourn that is too short indicates early reporting). Early reporting corresponds to a directional alternative; the main value of the one-sided test lies in threshold stability rather than in a multiplied power.
- **Same-state heartbeat.** The right-censored survival term $-\log\big(1 - F(\tau)\big)$, which alarms only when the sojourn is too long.
- **Timeout without message.** The same survival function, triggered by a timeout watchdog when the sojourn exceeds the 99.9% quantile of the group's log-normal model. This branch covers "the activity started but its completion event was suppressed", but not "the activity never started at all"; the latter requires a model of inter-activity gaps, which we do not provide.
- **Periodic polling.** The interval-censored term $\log\big[F(\tau + \Delta) - F(\tau)\big]$, which keeps quantization error out of the likelihood. The main experiments use an event-driven log, so this branch is not evaluated.

The nonconformity score of the timing channel is $-z$ (larger means more anomalous) rather than a parametric p-value, for reasons given in Section 4.5. The σ of manual stations can approach 2; for manual stations and for groups with $\sigma \geq 1$ (where $\rho^* > 90\%$ at $\alpha = 0.01$), the timing channel abstains. Overall detectability is governed by the most variable operation.

4.5 Conformal Calibration and Sequential Accumulation

Both statistical channels apply split conformal prediction [2, 31] on held-out normal samples: for each new message, the rank of its nonconformity score among the calibration scores gives a conformal p-value p_t. When calibration and deployment data are exchangeable, p_t is uniformly distributed on a benign stream, and deciding by $p_t \le \alpha/3$ guarantees a per-channel false-alarm rate of at most $\alpha/3$. The guarantee requires no Gaussian or Markov assumption, but it does require exchangeability [3]. In deployment data can only be split in time order, where exchangeability does not hold automatically; we therefore report the empirical false-alarm rate under a temporal split separately (Section 6.6). The lowest single-message false-alarm level attainable with a finite calibration set is about $1/(n+1)$, where n is the number of calibration samples; we do not set the single-message α below 10^{-2} and leave lower false-alarm budgets to the ARL_0 of CUSUM.

The nonconformity scores must preserve a monotone order in the tail, so both channels take raw scores as input, not parametric or randomized p-values. The timing channel takes $-z$: once a parametric p-value is truncated at 10^{-12}, benign mismatches and obvious early reports are compressed onto the same value and the tail order is lost. The structural channel takes $\tilde{P}$: a randomized probability integral transform would spread the many ties produced by Dirichlet smoothing over a uniform interval, giving a never-seen transition roughly an even chance of a p-value above α. The conformal p-value has its own randomization term, so ties are broken exactly once, at this step. Section 6.4 reports the measured effect of both choices.

Each channel runs a one-sided CUSUM [16] on its conformal p-values:

$$S_t = \max\big(0,\ S_{t-1} - \log p_t - c\big), \tag{1}$$

where $-\log p_t$ is the evidence contributed by message t; anomalies such as early reporting make p_t small and the evidence large. On a benign stream p_t is uniform and $-\log p_t$ is exponential with mean 1, so the reference value c must exceed 1 for the statistic not to grow without bound on benign data; we use $c = 1.5$. An alarm is raised when S_t crosses the threshold h, which is calibrated per path on the benign stream so that $\mathrm{ARL}_{0,i} = 1/\alpha_i$ ($3/\alpha$ with three paths). The statistic must be reset after an alarm; otherwise it stays above the threshold and every subsequent message raises an alarm. Every baseline that outputs continuous scores uses the same accumulation rule, so that gains are not misattributed to CUSUM itself.

4.6 Gated Update and Complexity

The predecessor state of each chain advances message by message. The duration model slowly updates the (μ, σ) of the corresponding group by an exponential moving average with forgetting factor 0.95; the $\tilde{P}$ of the structural channel is not updated online. Updates must be gated: a message may enter the baseline only if it passes every path and does not fall within the quarantine window (the 20 messages after the most recent alarm); otherwise an attacker could drag the duration baseline towards the forged value through sustained small early reports. Section 6.6 reports the measured effect of gating. The main-table and ablation experiments disable online updates and use frozen models; the gating and latency experiments follow the same decision protocol (three paths, $\alpha/3$ per path, one CUSUM per path) with only gated updates enabled.

The computational cost per message is $O(1)$: a constant number of table lookups, logarithms, and hard-constraint checks, independent of the numbers of states and devices on the hot path. Multiple devices can be processed in parallel chain by chain. Section 6.6 gives the constant factors.

5 Theoretical Analysis

5.1 Impossibility of Label-Only Detection for Label-Consistent Injections

Consider a class of detectors that use labels only, hereafter the *label-only detector family*: from the predecessor i it obtains a predictive distribution $\hat{\mathrm{p}}$, encodes the observation as a one-hot vector e_j, and compares $\delta(i,j) = \|\mathrm{e}_j - \hat{\mathrm{p}}\|_2$, or any quantity strictly decreasing in p_j, with an adaptive threshold. Assume that the predictive distribution satisfies $p_j = 0$ if and only if the transition $i \to j$ is forbidden by the feasibility mask F; that is, the predictor uses a smoothed estimate and produces no spurious zeros from insufficient samples (the Dirichlet posterior predictive of Section 4.3 satisfies this condition). Expanding

$$\delta(i,j)^2 = 1 - 2p_j + \|\hat{\mathrm{p}}\|^2 \tag{2}$$

shows that $\|\hat{\mathrm{p}}\|^2$ is constant over all j for a given predecessor i, so δ is merely a monotone transform of p_j and its geometric form adds no information.

Hence δ is a deterministic function of (i,j), regardless of whether the message is benign or injected. Let q_{ij} denote the true frequency with which j follows predecessor i in the benign stream, and FPR_i the false-alarm rate on benign messages that follow predecessor i.

Proposition 5.1 (Impossibility for the label-only detector family). *Suppose that the detector decides on an observation j following predecessor i based only on (i,j), that its decision is a monotone function of p_j, and that $q_{ij} = 0$ if and only if $i \to j$ is forbidden by* F. *Then: (i) under a zero-false-alarm constraint, the transitions that can be flagged are exactly those forbidden by* F*, so for label-consistent injections the family adds nothing beyond the mask; (ii) for injections that leave labels unchanged, the single-message decision on every message equals that on its benign counterpart; if the injection also preserves the message order, the sequential decisions are identical as well, and the net detection is zero regardless of α.*

Proof. (i) If some j with $q_{ij} > 0$ is flagged, every benign occurrence of j raises an alarm, so $\mathrm{FPR}_i \geq q_{ij} > 0$, contradicting the zero-false-alarm constraint; hence only transitions with $q_{ij} = 0$, that is, those forbidden by F, can be flagged. (ii) When labels are unchanged, the pair (i,j) of every message is unchanged and so is its decision; when the order is also unchanged, the score sequence coincides with the benign one message by message, the alarm positions are the same, and any detection comes from chance alarms only. □

The attacker's optimal evasive choice is $\arg\min_j \delta_j = \arg\max_j p_j$, that is, the next step that is most common in the log and thus closest to normal behaviour once forged.

The result does not depend on the particular threshold coefficient: every detector whose decision is a univariate function of the predicted probability belongs to this family. Proposition 5.1(ii) predicts that the first-order Markov ablation has zero net detection on the pure timing attacks A3/A5; Section 6.3 measures −0.01 and 0.00 after subtracting the chance-alarm floor, in agreement with the prediction.

5.2 Distributional Durations versus Clock Guards

A timed automaton assigns a clock guard $[\ell, u]$ to a transition (o', o): a sojourn time $\tau \in [\ell, u]$ is accepted and any other value is rejected [18, 21]. If an attacker shortens a normal sojourn τ to $(1-\rho)\tau$ and $(1-\rho)\tau$ still lies in $[\ell, u]$, the single-step decision is "normal". Because the guard is a binary decision, repeated sojourns that are slightly fast in the same direction are not accumulated.

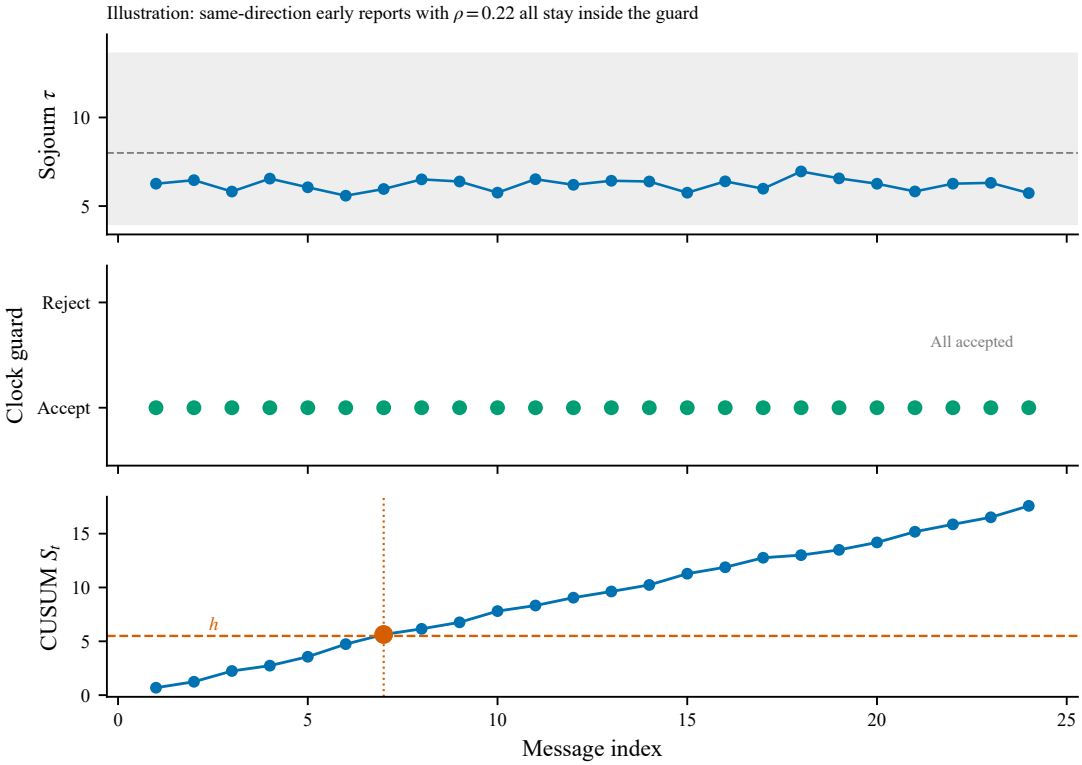


Fig. 4. Clock guard versus distributional residual (illustration, not an evaluation trace). The grey band is the interval guard; every same-direction early report is accepted, yet CUSUM accumulates and crosses h.

The semi-Markov channel outputs the continuous residual $z = (\log \tau - \mu)/\sigma$. Early reporting yields $\mathbb{E}[z] = \log(1-\rho)/\sigma < 0$, so the corresponding conformal p-values are systematically small and the mean of the evidence $-\log p_t$ increases. On a benign stream $\mathbb{E}[-\log p_t] = 1 < c$; only when the shift is large enough that $\mathbb{E}[-\log p_t] > c$ does the CUSUM statistic drift upwards and cross h, even if no single observation crosses the single-message threshold on its own. For small shifts the statistic has no positive drift and crosses the threshold only by chance fluctuation, which matches the low detection at small ρ in Table 8. The difference therefore lies not in whether sojourn times are modelled but in the *form of the evidence*: an interval guard maps early reports inside the interval to zero, whereas a distributional residual maps them to additive increments. Figure 4 illustrates this: all observations fall inside the guard, the binary channel stays silent, and CUSUM still crosses the threshold. A3 is the typical case of this difference: its labels are entirely legal and a timed automaton accepts it, yet a sequential semi-Markov test can still detect it.

5.3 Critical Relative Advance

This subsection gives the critical fraction by which a forged sojourn can be shortened under a single-message test. Consider a left-sided single-message test that raises an alarm when the residual falls below the threshold $-z_{1-\alpha}$. When the attacker shortens the sojourn to $(1-\rho)\tau$, the whole residual distribution shifts to the left; when the shift places the residual mean exactly on the threshold, a single forged message is detected with probability $1/2$. An attacker who

wants each forgery to evade the test with probability greater than one half therefore cannot exceed this critical relative advance, which we denote by ρ^*. It is not a strict upper bound on evasion: forgeries with $\rho < \rho^*$ are still detected, with probability below one half, and sequential accumulation raises detection further.

Proposition 5.2 (Critical relative advance). *Let normal sojourns satisfy* $\log \tau \sim \mathcal{N}(\mu, \sigma^2)$*, and consider a left-sided test of level* α *with threshold* $-z_{1-\alpha}$*. The attack shortens the observed duration to* $(1-\rho)\tau$*. The relative advance at which the post-attack residual mean lies exactly on the threshold is*

$$\rho^*(\alpha, \sigma) = 1 - e^{-\sigma z_{1-\alpha}}. \tag{3}$$

If the test acts on the mean of k *independent forged residuals, whose standard deviation shrinks as* $1/\sqrt{k}$*, then*

$$\rho_k^*(\alpha, \sigma) \approx 1 - e^{-\sigma z_{1-\alpha}/\sqrt{k}}. \tag{4}$$

Proof. The log-domain shift is $\Delta = \log(1-\rho)$, which moves the standardized mean by Δ/σ. Setting $\Delta/\sigma = -z_{1-\alpha}$ gives $\rho = 1 - e^{-\sigma z_{1-\alpha}}$. Averaging k independent residuals replaces σ by $\sigma/\sqrt{k}$, which yields Eq. (4). □

Eq. (3) involves only the timing variability and the false-alarm budget, not a system matrix. It instantiates the fundamental limits of stealthy attacks [19] in the discrete-event setting: the form of the result is not new; what is new is that the instantiated quantity is the relative advance rather than the energy of a continuous state deviation. Eq. (4) gives the tightening under repeated injection; our experiments validate single injections only, and we do not present the $\sqrt{k}$ shrinkage as a validated result.

The one-half point above refers to a *single* Gaussian distribution. The σ of the operations on a production line are highly heterogeneous, and low-variability groups are detected at relative advances far below the global ρ^*; consequently the overall detection curve after mixing over operations (hereafter the mixture power curve) can lie clearly above $1/2$ already for $\rho < \rho^*$. Section 6.5 validates how well this mixture curve agrees with the measurements. Figure 5 shows ρ^* as a function of σ: the global $\sigma = 0.236$ corresponds to 42.2%; as the σ of manual stations approaches 2, ρ^* tends to 1 and the test loses its discriminative power, so the detectability of the line as a whole is governed by its most variable operation. Section 6.5 validates the result with a single-channel test at $\alpha = 0.01$; in the three-path configuration the timing channel receives $\alpha/3$, which corresponds to $\rho^* = 47.3\%$ at $\sigma = 0.236$.

6 Experiments and Results

6.1 Experimental Setup

The main data are the Fischertechnik smart-factory IoT event log released by the teaching factory of the University of Trier [22]. We chose this log because it is one of the few public activity logs that provide both a BPMN reference model and the assignment–execution–completion lifecycle, not because its scale is representative. Datasets of 1 Hz process variables such as HAI cannot instantiate assignments and process activities [32], and surveys of industrial-control testbeds and public datasets are likewise dominated by process variables and network traffic [5, 6]; we therefore report no cross-dataset results. After cleaning, the full log contains 3,062 activities, and the dozen or so devices on the line are all service endpoints that are invoked.

The diversity within the log is used for stress tests and does not substitute for a second external dataset: 16 BPMN models are released with the data; the official error-containing version (with NTP clock drift and duplicated and missing records) is used for false-alarm stress tests; and the temporal test fold contains five workflows unseen in the training fold. The feasibility mask F is derived automatically from the BPMN by the rules of Section 4.2.

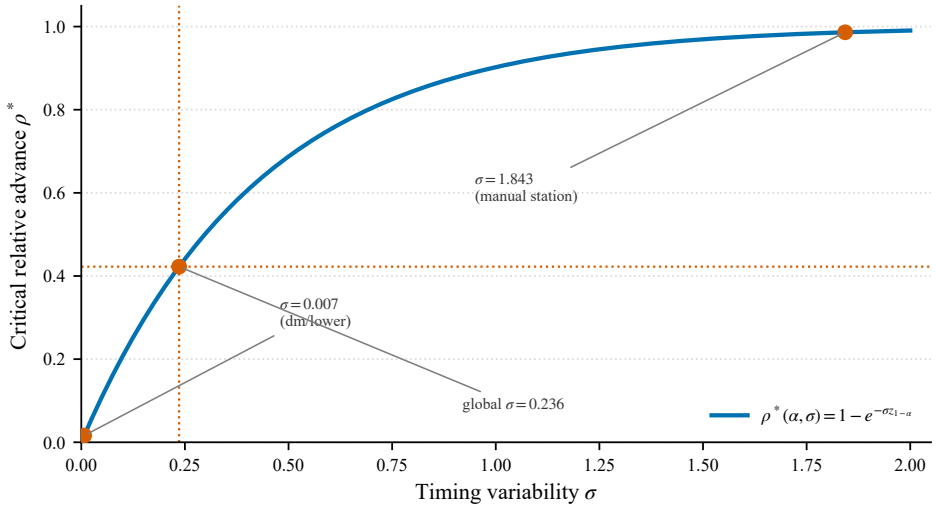


Fig. 5. ρ^* as a function of σ for a one-sided test at $\alpha = 0.01$. Marked points are a low-variability operation, the globally weighted σ, and a manual station.

Public activity logs contain no injection labels [5, 6]. We map replay, injection, and dropping in the Dolev–Yao model to the fields of this log, with the rules given in Table 2; the injector is released with the code. The data are split in time order into disjoint fit (1,951 activities), calibration (603), and test (508 benign events) folds, and injections are applied only to the test fold. The main table uses an injection rate of 20%, and we also sweep 5%/10%/20%/40%; the default relative advance is $\rho = 0.30$. The sample structure is 1 log × 1 temporal split × 6 attack classes × 3 injection seeds; the injection seeds $(0, 1, 2)$ determine only the injection positions and do not re-split the data, and the detector's fitting seed is fixed at 0. Over the three seeds, each attack class has 297 injections (290 for A1, 303 for A2, 201 for A6), and under a given seed all methods face the same attacked stream. The relative-advance sweep of Section 6.5 additionally uses a calibration set sampled with stratification by group ($n_{\text{cal}} = 701$), so that every (device, operation) group has enough calibration samples. Detection delays are archived per injection instance, and box plots include only instances detected within the budget. All experiments run on a single machine (Intel Xeon W-1270, 64 GB of memory) with Python 3.7.6, NumPy 1.18.1, and SciPy 1.4.1.

All three main baselines are reimplemented from the structural constraints of the original papers rather than from official code: neither BUTLA nor TABOR comes with a released implementation that accepts this log format, and the hidden semi-Markov model (HSMM) is a model class rather than an industrial detector. Table 3 lists the structure retained and the approximations made. Every method that outputs continuous scores uses the same CUSUM (reference value $c = 1.5$, delay budget of 10 messages, reset after alarm). Each method's false-alarm budget is split evenly across its sub-detectors, and a sub-detector with budget α_i calibrates its threshold h for $\text{ARL}_0 = 1/\alpha_i$: the label-only ablation has 1 path, B1/B3 have 2 each, and B2 and our method have 3 each. To compare methods at the same false-alarm level, the thresholds and CUSUM limits in the main table are calibrated on the uninjected version of the test fold, which aligns the empirical false-alarm rates with α. This setting uses neither attack labels nor the benign events inside the attacked streams, but it borrows the benign distribution of the test fold and is thus an idealized calibration. In deployment thresholds can come only from the calibration fold, so we also rerun every method with the calibration fold as the

Table 3. Reimplementation protocol for the main baselines. The comparison is against the structural constraints of the original papers, not against official binaries.

Method	Retained	Approximation on this log
B1 BUTLA-style [21]	One automaton learned per component; no cross-component check; duration as a deviation tolerance	Component taken as device; tolerance replaced by the empirical quantile at the same α, so that differences in false alarms are not counted as differences between methods
B2 TABOR-style [18]	Within-stage dependencies; no assignment (command) data	Stage taken as device; within-stage joint as $P(o)\prod P(v \mid o)$ (compared with the saturated joint below the table)
B3 HSMM [33]	Semi-Markov model with explicit durations	Observations are the operation and $\log\tau$; $K \in \{2,3,4,6\}$, with $K=6$ selected by held-out likelihood

reference stream; the false-alarm rates are then no longer aligned, and the detections are reported together with the test-fold false-alarm rates in Section 6.3. Our method, like every baseline, is fitted on the fit fold only. In the main table, the raw scores of our three paths and the baseline scores go through the same decision procedure (empirical p-values against the reference stream and the CUSUM above), bypassing the frozen conformal calibrator, so that two threshold mechanisms are not mixed in the comparison; per-channel empirical false-alarm rates with the conformal calibrator are given in Section 6.6. The label-only first-order Markov likelihood is an ablation that removes the time dimension, not an independent prior method. The reference-model conformance proxy (alarm on any violation of F) makes the same decisions as the hard-constraint layer and is not listed separately in the main table; its single-message results are the hard-constraint column of Table 4.

The within-stage structure learned in the original paper lies between the naive factorization and the saturated joint distribution $P(o, v_1, \ldots, v_m)$. Replacing the within-stage distribution of B2 with the saturated joint under the main-table protocol changes its six-class mean from 0.21 to 0.22 and its A3 result from 0.10 to 0.08; our method remains above this variant on all six attack classes.

When the rule "any alarm in the window counts as a detection" is applied within the delay budget, random false alarms in the window yield some detection rate even for a detector with no real detection capability. This chance-alarm floor is $1-(1-\mathrm{FPR})^{B+1}$, where $B=10$ is the delay budget. All main results in the tables have this floor subtracted, and values without the subtraction are not quoted. Per-message detection rates are limited by the length of the reference stream and are not used as main results at this scale.

Methods are compared with the injection instance as the pairing unit: under a given seed all methods face the same attacked stream, and for each injection we record whether it is detected within the delay budget. For each attack class we apply a two-sided exact McNemar test to the difference between our method and each competitor, and we compute 95% confidence intervals for the difference in detection rate from 2,000 bootstrap resamples over injection instances. The 6 attack classes and 4 competitors give 24 comparisons, corrected by the Holm method. The test is applied to raw detections without subtracting the chance-alarm floor; treating injections within the same attacked stream as independent is an approximation.

Table 4. Single-message detection rates of the three production paths on the six injection classes, with the false-alarm rate on the same benign events in parentheses. Each path decides on its own at $\alpha = 0.01$ with frozen models; test fold of 508 events, injection rate 20%, mean over three injection seeds. The hard-constraint value of 0.06 on A2 is not a purely benign false alarm but a consequence of the cascaded check: the benign event following an injection has the forged operation as its predecessor and is therefore rejected by the binary reachability check. Its deployment meaning is "attack detected but attributed to the following event", so it is reported separately from the purely benign false-alarm rates of Section 6.3.

ID	Class	Hard constraint F	Structural	Timing
A1	Naive replay	0.22 (0.00)	0.21 (0.02)	0.03 (0.02)
A2	Physically infeasible injection	0.98 (0.06)	0.29 (0.02)	0.00 (0.02)
A3	Early-completion replay	0.00 (0.00)	0.02 (0.01)	0.43 (0.02)
A4	State mimicry (known model)	0.03 (0.00)	0.01 (0.01)	0.00 (0.02)
A5	Gradual drift	0.00 (0.00)	0.01 (0.01)	0.48 (0.03)
A6	Message suppression	0.00 (0.00)	0.16 (0.01)	0.04 (0.02)

6.2 Single-Channel Coverage Diagnosis

Before the end-to-end comparison, Table 4 gives the single-message detection rate of each of the three paths, to show which path is responsible for which attack. A2 is rejected by the unary capability set, and the timing channel is insensitive to it. A3/A5 leave labels unchanged and only compress sojourn times; the hard-constraint layer and the structural channel detect almost nothing, and the timing channel is the only one with substantial detection. A1 keeps historical durations, so the timing channel gains nothing and detection is carried by the hard-constraint layer and the structural channel. A4 inserts the most probable next step according to the transition model, and the single-message detection of all three paths is close to the false-alarm level. A6 deletes an entire activity that has not yet started, so the timeout watchdog is never triggered, and the structural channel sees only a weak signal downstream. Figure 6 plots the same table as a heat map of the net lift over false alarms.

The numbers in Table 4 cannot be compared directly with Table 5: the former is a single-channel, single-message diagnosis without budget splitting and without subtracting the chance-alarm floor; the latter is the end-to-end net detection of three parallel paths with $\alpha/3$ per path, a delay budget of 10 messages, and the chance-alarm floor subtracted. For example, the single-message timing detection on A3 is 0.43, whereas Table 5 reports 0.18; the two are not in conflict.

6.3 Comparison with Timed Automata and Semi-Markov Models

Table 5 gives the net detection rates at $\alpha = 0.01$. Our method averages 0.48 over the six attack classes, against 0.17/0.21/0.11 for B1/B2/B3. We discuss three groups of attacks in turn.

First, pure timing attacks. A3/A5 leave labels unchanged, and the label-only ablation scores -0.01 and 0.00 after subtracting the chance-alarm floor; its score stream coincides with the benign stream message by message, consistent with Proposition 5.1(ii). On A5 our method reaches 0.87, the largest margin in the table; timed-automaton baselines also capture cumulative drift (0.59/0.48), but remain below the distributional channel with its left-tail test per (device, operation). On A3 our method reaches 0.18 ± 0.01, against 0.10 ± 0.03 and 0.10 ± 0.04 for B1/B2. In the per-injection paired test, the difference in detection rate is 0.067 against B1 (95% interval $[0.020, 0.114]$, Holm-corrected $p = 0.031$) and 0.064 against B2 ($[0.010, 0.118]$, corrected $p = 0.096$, not significant). The gain is modest, and it arises because sporadic early reports can still be accumulated under distributional residuals. B3 reaches only 0.06 on A3, which shows

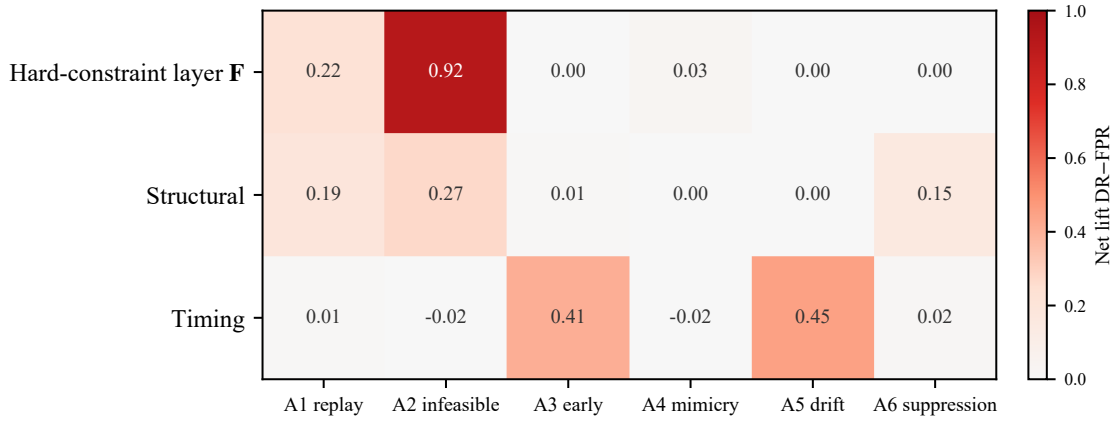


Fig. 6. Net lift of the three production paths over the benign false-alarm rate (DR−FPR). A2 is covered by the hard-constraint layer, only the timing channel gives a substantial lift on A3/A5, and A4 has no dedicated path.

that the gain does not come merely from the model-class difference of "semi-Markov model versus interval guard": the duration kernel of HSMM is tied to hidden states, and an early report with an unchanged label is absorbed as a slightly shorter stay.

Second, replay and infeasible injection. On A1/A2 our method reaches 0.55/0.95, higher than all three main baselines (paired test, Holm-corrected $p < 10^{-20}$ in every case). A2 is covered by the unary capability set; the within-stage dependencies of B2 cannot substitute for the reference model. B2 and B1 perform similarly on most attack classes: within-stage dependencies carry little information on this log, and the extreme scores that B2 assigns to "this device has never executed this operation" already occur in the benign test fold (the unseen combinations described in Section 4.2), whereas the reference model does not reject these combinations.

Third, attacks that change the number of events. On A4/A6 our method reaches 0.17/0.14, below the 0.23/0.28 of the label-only ablation (Holm-corrected $p = 0.001$ and $p < 10^{-6}$). These two attacks are handled mainly by the structural channel; with the even three-way split it receives only $\alpha/3$, whereas the label-only ablation keeps all of α on its single path, and the budget split is an explanation consistent with the gap. We report this gap as a cost of the multi-channel design, not as a main conclusion. At $\alpha = 0.05$ our method (0.18/0.19) is still below the label-only ablation (0.41/0.45) on A4/A6, so the disadvantage is not caused by insufficient calibration resolution at $\alpha = 0.01$; the ranking is not fully preserved, however, since on A1 our method also falls below the label-only ablation at this level (0.47 versus 0.61).

The EM likelihood of B3 converges monotonically and its duration kernel is non-degenerate, so it serves as a baseline directly comparable to "timing plus structure"; its mean is only 0.11, and only 0.31 on A2, consistent with the absence of feasibility constraints in a hidden-state model. When thresholds are instead calibrated with the calibration fold as the reference stream (Section 6.1), every method exceeds the nominal sequential false-alarm rate of 0.01 on the test fold: B1 0.043, B2 0.138, B3 0.028, label-only 0.018, ours 0.030. Our method thus also fails to maintain the nominal false-alarm rate under a temporal split, consistent with the per-channel results of Section 6.6. As the false-alarm rates are no longer aligned, the net detections serve only as a qualitative reference: our method is still the highest on A3/A5 (0.45/0.71) and falls below the label-only ablation on A2 (0.72 versus 0.76). Taken together, the advantage on duration forgery holds against all three baselines on A5, and on A3 it is significant only against B1/B3.

Table 5. Net detection rates at $\alpha = 0.01$ with a delay budget of 10 messages and the chance-alarm floor subtracted. Mean ± population standard deviation over three injection seeds, 201–303 injections per attack class in total. Label-only is the Markov ablation. Sequential false-alarm rates in the attacked streams (nominal 0.01): B1 0.006, B2 0.006, B3 0.008, label-only 0.008, ours 0.004. Paired tests (exact McNemar, Holm correction over 24 comparisons) are reported in Section 6.3; results with thresholds calibrated on the calibration fold are given in Table 6.

Attack	B1 BUTLA	B2 TABOR-style	B3 HSMM	Label-only	Ours
A1 Naive replay	0.04±0.01	0.04±0.01	−0.01±0.03	0.48±0.04	0.55±0.04
A2 Infeasible injection	0.37±0.05	0.67±0.03	0.31±0.04	0.76±0.04	0.95±0.00
A3 Early-completion replay	0.10±0.03	0.10±0.04	0.06±0.04	−0.01±0.02	0.18±0.01
A4 State mimicry	−0.04±0.01	−0.01±0.01	0.02±0.01	0.23±0.04	0.17±0.03
A5 Gradual drift	0.59±0.09	0.48±0.07	0.25±0.08	0.00±0.12	0.87±0.06
A6 Message suppression	−0.05±0.01	−0.01±0.03	0.05±0.02	0.28±0.08	0.14±0.06
Mean	0.17	0.21	0.11	0.29	0.48

Table 6. Net detection rates and test-fold sequential false-alarm rates with thresholds calibrated on the calibration fold (603 events) ($\alpha = 0.01$, delay budget of 10 messages, mean ± population standard deviation over three injection seeds). The false-alarm rates of the methods are no longer aligned; the chance-alarm floor is subtracted according to each method's measured false-alarm rate, and the results serve only as a qualitative reference.

Attack	B1 BUTLA	B2 TABOR-style	B3 HSMM	Label-only	Ours
A1 Naive replay	−0.05±0.06	−0.23±0.04	−0.03±0.01	0.54±0.05	0.45±0.02
A2 Infeasible injection	0.26±0.08	0.20±0.00	0.53±0.01	0.76±0.03	0.72±0.00
A3 Early-completion replay	0.28±0.03	−0.13±0.05	0.08±0.06	0.00±0.01	0.45±0.04
A4 State mimicry	−0.12±0.04	−0.32±0.03	0.13±0.07	0.30±0.04	0.16±0.02
A5 Gradual drift	0.55±0.02	0.15±0.03	0.24±0.08	−0.02±0.22	0.71±0.02
A6 Message suppression	−0.12±0.03	−0.36±0.05	0.12±0.03	0.36±0.06	0.17±0.09
Mean	0.13	−0.11	0.18	0.32	0.44
Sequential FPR	0.043	0.138	0.028	0.018	0.030

Figure 7 plots Table 5 as grouped bars; the label-only ablation stays near zero on A3/A5, reflecting the blind spot created by removing the time dimension. Figure 8 shows box plots of the detection delay in the same evaluation, counting only injections detected within the delay budget; undetected instances do not enter the quantiles. Table 7 gives the alarm intervals on the purely benign stream, counted separately from the sequential false alarms in the attacked streams. On the 508 benign events each method raises only 2–4 alarms (ours: 2, i.e. a sequential false-alarm rate of 0.004), so the interval statistics are only a reference and are insufficient to estimate ARL_0. Figure 9 sweeps the injection rate from 5% to 40%. On A2 our method is the highest at every injection rate, but its lead narrows as the rate increases. On A5 our method is slightly below B1 only at 10% (0.67 versus 0.69). On A3 our method exceeds the timed-automaton baselines only at 20% and 40%, not at 5% and 10%. On A4 it is below the label-only ablation at every injection rate, and on A6 it exceeds the latter only at 5%.

6.4 Ablation

The ablation follows the decision procedure of the main table at $\alpha = 0.01$ and *fixes the false-alarm budget of each path*, to avoid the confound that removing one path would give more budget to the others. The full configuration averages 0.48 over the six attack classes. Removing the timing channel lowers the mean by 0.18 and drives A3/A5 to zero ($0.18 \to -0.01$, $0.87 \to -0.02$). Without the timing channel only the hard-constraint layer and the structural channel remain; both read labels only and thus detect nothing of pure duration attacks, which corroborates the zero detection of the label-only ablation on the same two classes in Table 5. Removing the hard-constraint layer lowers the

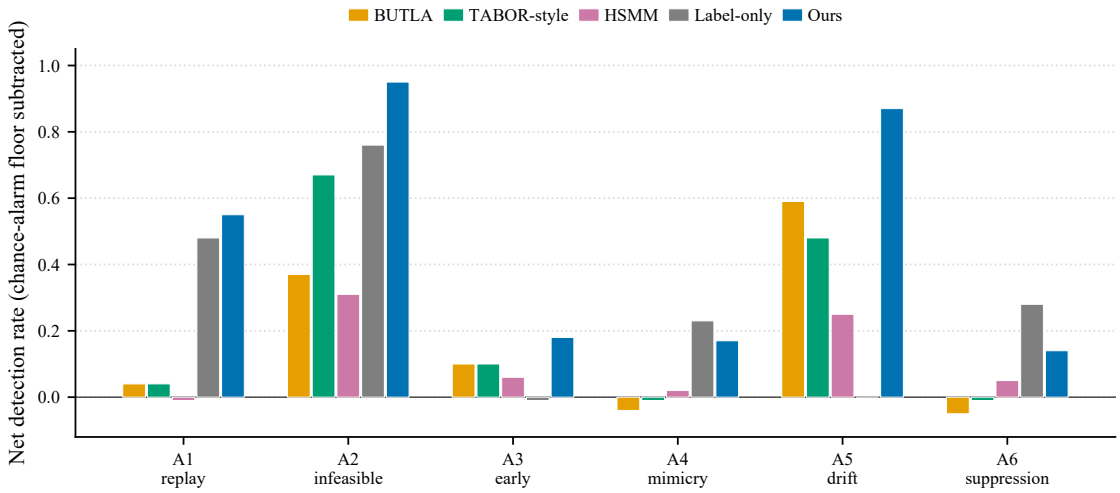


Fig. 7. Net detection rates at $\alpha = 0.01$ with a delay budget of 10 messages and the chance-alarm floor subtracted. Label-only is the Markov ablation, not an independent prior method.

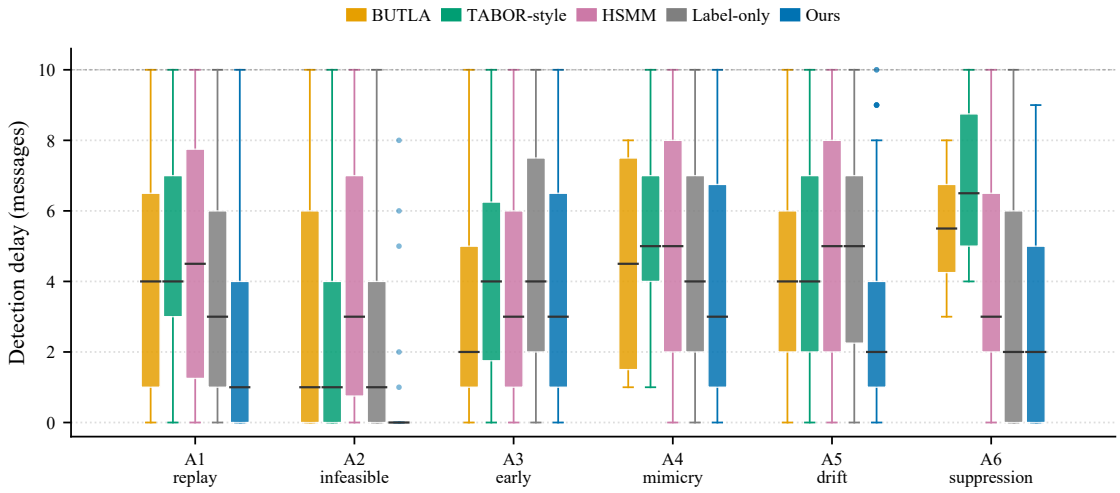


Fig. 8. Detection delays (in messages) of each method on the six attack classes at $\alpha = 0.01$ with a delay budget of 10 messages. Boxes include only detections within the budget; for undetected injections see the net detection rates.

mean by 0.08, with the drop concentrated on A2 (0.95 → 0.70) and A1 (0.55 → 0.42). Removing the structural channel lowers the mean by 0.05, with the drop concentrated on A6 (0.14 → −0.02) and A4 (0.17 → 0.08). At $\alpha = 0.05$ the three drops are 0.16, 0.04, and 0.03, in the same order. Of the three paths, only the timing channel is irreplaceable: it is the sole source of detection on A3/A5. Figure 10 shows the per-class comparison and the change in the mean.

The role of conformal calibration shows on the false-alarm side: on the stratified calibration set of Section 6.5, replacing the one-sided threshold by a Gaussian quantile raises the empirical false-alarm rate on the test set from 0.0126 to 0.0168 (nominal 0.01).

Table 7. Alarm intervals on the purely benign stream of the test fold (508 events). The three seeds share the same benign stream, and alarms are counted on that stream; intervals include the stretch from the start of the stream to the first alarm. Alarms are too few to estimate ARL_0.

Method	Median interval	Mean interval	Alarms
B1 BUTLA	21	152	3
B2 TABOR-style	59	158	3
B3 HSMM	119	115	4
Label-only	56	83	4
Ours	218	218	2

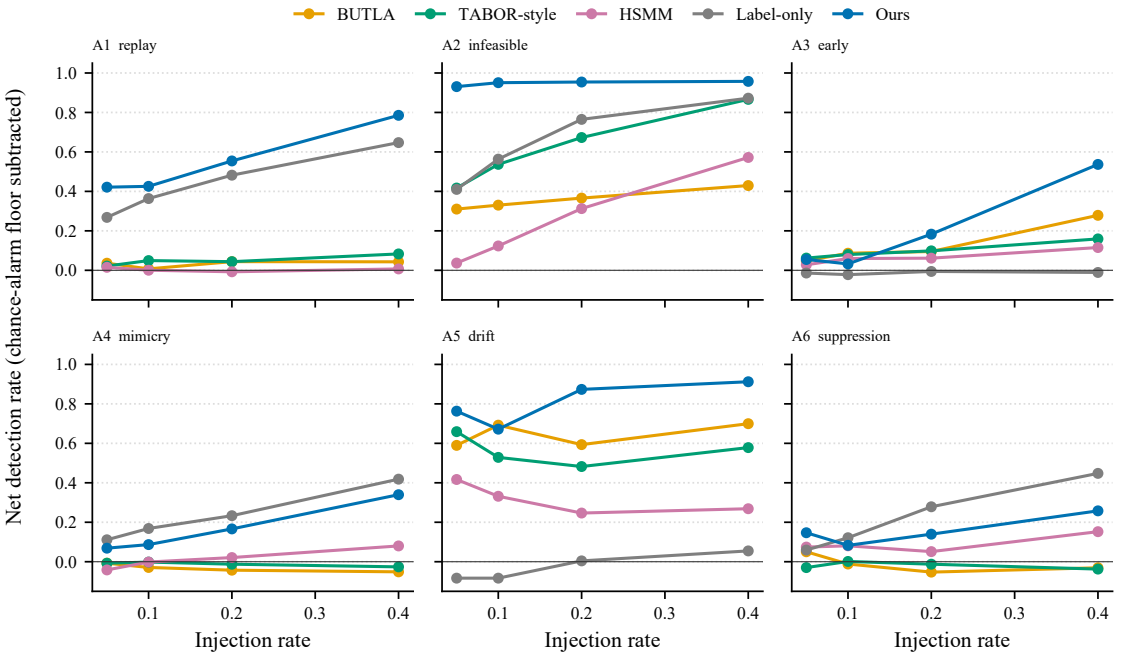


Fig. 9. Net detection rate versus injection rate (5%–40%). One panel per attack class, showing whether the conclusions break down as attack density increases.

Several design choices of the method were also settled by measurement. If the timing channel took a truncated parametric p-value instead of $-z$, the single-message detection on A3 would fall from 0.43 to 0.06 (on the same basis as Table 4). Material-token conservation was considered as a candidate channel, but its benign violation rate is too high and its net end-to-end contribution is zero (Section 7.2); it is therefore excluded from the production configuration and consumes no false-alarm budget.

6.5 Relative-Advance Sweep

We apply the injection $\tau \mapsto (1-\rho)\tau$ to real sojourn times. The ρ^* of Proposition 5.2 uses the global $\sigma = 0.236$ of the fit fold (not conditioned on covariates), giving $\rho^* = 42.2\%$ for a one-sided test at $\alpha = 0.01$. The theoretical values in

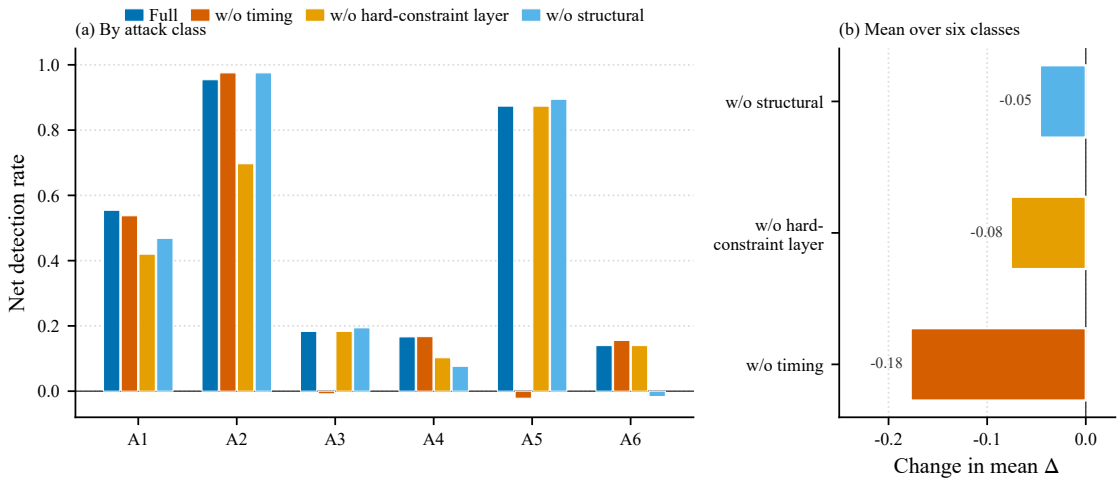


Fig. 10. Ablation of the three production paths ($\alpha = 0.01$, fixed per-path false-alarm budget, mean over three injection seeds, chance-alarm floor subtracted). Removing timing drives A3/A5 to zero; the drop from removing the hard-constraint layer is concentrated on A2/A1, and that from removing the structural channel on A6/A4.

Table 8. Relative-advance sweep for a one-sided test at $\alpha = 0.01$ (stratified calibration set $n_{\text{cal}} = 701$, 715 test events). Shift $\Delta = \log(1-\rho)$; $\bar{\sigma}$ is the mean timing variability of the test groups (about 0.24); the theory column is the mixture prediction over operations.

ρ	$\lvert\Delta\rvert$	$\lvert\Delta\rvert/\bar{\sigma}$	Measured DR	Theory
0.05	0.051	0.21	0.062	0.057
0.10	0.105	0.44	0.073	0.132
0.15	0.163	0.68	0.206	0.239
0.20	0.223	0.93	0.291	0.338
0.30	0.357	1.49	0.516	0.537
0.40	0.511	2.13	0.740	0.745
0.50	0.693	2.89	0.899	0.878

Table 8 and Figure 11 are the mixture prediction over operation groups (Section 5.3); their mean absolute deviation from the measurements over the whole range is 0.027 (95% interval $[0.018, 0.035]$ from 2000 bootstrap resamples drawn independently from the calibration and test folds; 0.012–0.031 over 20 fold-split seeds). At $\rho = 0.30 < \rho^*$ the measured single-message detection already reaches 0.52, and at $\rho = 0.40 \approx \rho^*$ it is 0.74, whereas a single Gaussian would give only $1/2$ at ρ^*; this is exactly what the mixture power curve predicts.

CUSUM turns weak signals into usable detection capability. In this experiment it is driven by parametric left-tail p-values on the stratified calibration set, with a threshold corresponding to $\text{ARL}_0 = 500$ and a delay budget of 40 messages. At $\rho = 0.15$ the single-message detection is 0.21 and rises to 0.55 after accumulation, with a median delay of 15 messages; at $\rho = 0.20$ it rises to 0.85; at $\rho = 0.30$ it is 0.93, with a median delay of 7 messages (Figure 12). Detection saturates at about 0.96; the undetected remainder comes from groups with very large σ, consistent with overall detectability being governed by the most variable operation.

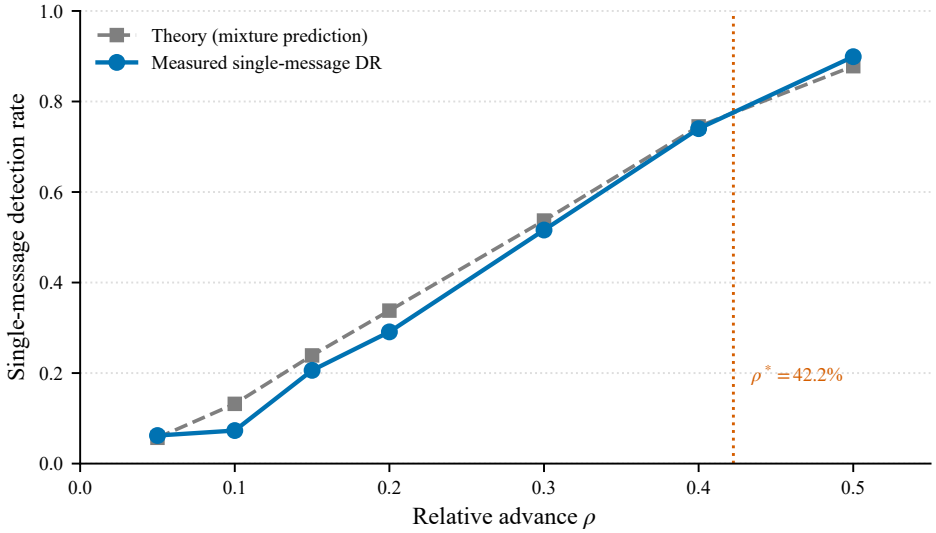


Fig. 11. Single-message detection rate versus relative advance. The theory curve is the group-mixture prediction; the vertical line marks $\rho^* = 42.2\%$ at the global $\sigma = 0.236$.

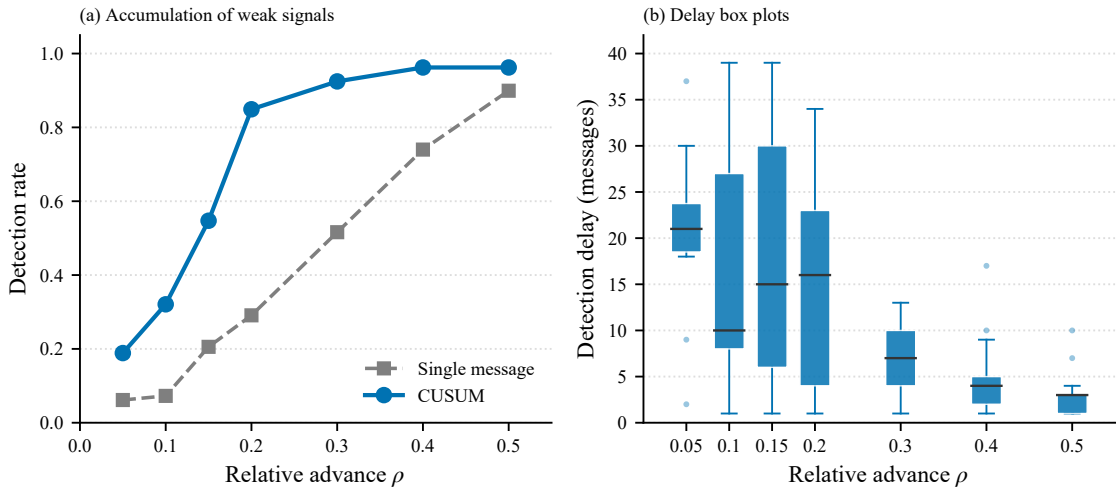


Fig. 12. Power of CUSUM relative to the single-message threshold, and box plots of the detection delay. Driven by parametric left-tail p-values on the stratified calibration set, threshold for $\mathrm{ARL}_0 = 500$, delay budget of 40 messages, 53 test substreams. Boxes include only injections detected within the budget; undetected injections are censored (see the net detection rates in the main table).

6.6 Deployment Conditions and Scope

The preceding subsections present the main results. This subsection reports four sets of data that must be stated before deployment.

First, false alarms under a temporal split. The conformal guarantee relies on exchangeability, whereas real deployment can only split data in time order. Table 9 splits fitting and testing 75%/25% by case (about 636 test messages):

Table 9. Per-channel empirical conformal false-alarm rates. Split 75%/25% by case (about 636 test messages); the random fold uses 5 random splits, the temporal fold 5 seeds under the same split; mean ± standard deviation.

Channel	Random fold		Temporal	
	α=0.05	α=0.01	α=0.05	α=0.01
Timing	0.051±0.007	0.006±0.005	0.098±0.001	0.024±0.001
Structural	0.048±0.013	0.008±0.006	0.064±0.003	0.012±0.001

the random folds use 5 random splits; the temporal fold admits only one split, and its 5 seeds change only the randomization in fitting and conformal calibration, hence the very small standard deviations. The table shows that under random splits the empirical false-alarm rates stay close to their nominal values, whereas under the temporal split at $\alpha = 0.01$ the timing channel rises to 0.024 and the structural channel to 0.012. Among the 16 workflows released with the data, the median Jaccard similarity of their transition sets is only 0.324, i.e. the transition structures of different workflows differ substantially; new product types must therefore be recalibrated, or calibrated in strata by type. These values are per-channel single-message false-alarm rates and are computed differently from the sequential false-alarm rate of 0.004 in the attacked streams of Section 6.3; if the sequential thresholds are also taken from the calibration fold, the sequential false-alarm rate of our method is 0.030 (Section 6.3), in the same direction as the per-channel results.

Second, stress tests within the log (Table 10). The timing false-alarm rate on the official error-containing version is 0.112; it stems from benign timing anomalies caused by data-acquisition defects, not from attacks. On unseen workflows the timing false-alarm rate is 0.067. The hard-constraint layer has false-alarm rates of 0.003 and 0 in these two cases. Product changeover is a distribution shift: the timing channel must be recalibrated per product type, whereas the hard-constraint layer need not be.

Third, recalibration of the duration kernel. Artificially doubling the σ of the fit fold lowers the single-message and CUSUM detection at $\rho = 0.30$ from 0.52 and 0.94 to 0.22 and 0.45; at $\sigma \approx 0.96$ only 0.05 of single-message detection remains, so the scope is delimited by the most variable operation. Table 11 further demonstrates on a self-built simulated production line that the duration kernel must be recalibrated per scenario. The three scenarios share the operation vocabulary and the feasibility mask F: scenario I has 2 automated guided vehicles and 1 robot arm with $\sigma = 0.12$ and short travel distances; scenario II has 4 automated guided vehicles and 2 robot arms with $\sigma = 0.12$ and long travel distances; scenario III has the layout of scenario I with $\sigma = 0.35$. On scenario II, A3 detection is 0.56 with the duration kernel calibrated on the scenario itself and drops to 0.06 with the kernel of scenario I (the scenario has only 18 A3 injections, i.e. 10/18 versus 1/18); transfer in the reverse direction (II→I) drives the timing false-alarm rate to 0.493. In every setting the hard-constraint layer has zero false alarms, because all scenarios share the same F.

Fourth, gating and computational cost. Under 200 sustained early reports with $\rho = 0.30$, disabling the gate lets 26.4% of the forged advance enter the baseline (all 200/200 updates take effect), whereas enabling it reduces this to 1.4% (3/200). The median per-event processing latency on a single core is 123 μs ($p_{95} = 191$ μs, $n = 1{,}524$); when the scale is increased by replicating device names, the median latency does not grow with scale. We did not measure on a dedicated controller.

Table 10. Stress tests within the same Trier log ($\alpha = 0.01$, no injection). The error-containing version is released by the original authors; seen and unseen workflows are obtained by splitting the temporal test fold.

Setting	n	Hard constraint	Timing	Structural
Cleaned temporal test fold	508	0.000	0.024	0.012
Official error-containing full log	2,778	0.003	0.112	0.011
Seen workflows	329	0.000	0.000	0.012
Unseen workflows	179	0.000	0.067	0.006

Table 11. Recalibration of the duration kernel on self-built simulated scenarios ($\alpha = 0.01$, $\rho = 0.30$). The first three columns are the false-alarm rates of each path, and the A3 column is the single-message detection of the timing channel; "I→II" means calibrated on scenario I and tested on scenario II. Each scenario has 69–90 benign events and 13–18 A3 injections.

Setting	Hard constraint	Timing	Structural	A3
Scenario I	0.000	0.000	0.014	0.462
Scenario II	0.000	0.000	0.011	0.556
Scenario III	0.000	0.000	0.014	0.308
I→II	0.000	0.000	0.000	0.056
I→III	0.000	0.101	0.000	0.385
II→I	0.000	0.493	0.000	0.692

7 Conclusion

7.1 Conclusions and Contributions

We have formalized injection detection on discrete activity events with a lifecycle. On the theoretical side, we have shown in a numbered proposition (Proposition 5.1) that the detector family which thresholds only the predicted probability of labels adds nothing beyond the feasibility mask for label-consistent injections under a zero-false-alarm constraint, and we have derived the critical relative advance $\rho^*(\alpha, \sigma)$.

On the methodological side, we build a hard-constraint layer from the reference-model feasibility mask (zero violations on the cleaned normal data) and a timing channel from distributional sojourn times grouped by (device, operation); the two statistical channels are calibrated by conformal prediction and accumulated by CUSUM tests. Compared with decomposed timed automata and within-stage dependency models, our method differs in two respects: distributional durations allow early reports inside the guard to be accumulated, and feasibility comes from the reference model rather than being learned from logs.

The experiments show that physically infeasible injection is covered by the feasibility mask (net detection 0.95); on gradual drift our method achieves 0.87, while the label-only ablation scores zero; on early-completion replay at a 20% injection rate it achieves 0.18, higher than BUTLA and not significantly different from the TABOR-style baseline; and the mixture power curve predicted by the critical-relative-advance model agrees with the measurements. These results indicate that, for lifecycle activity events in industry, reference-model feasibility and distributional sojourn times are two kinds of evidence missing from label-only detection; the disadvantage on A4/A6 shows that they cannot replace workflow structure.

7.2 Limitations

(1) State mimicry and message suppression lack a dedicated detection path. On A4/A6 our detection is below that of the label-only ablation. There are two possible causes whose effects have not been separated: after the three-way split the structural channel receives only $\alpha/3$, and its state is the operation name, which is insensitive to the device. This limitation qualifies the comparison of "six-class means" but does not affect the claims about A2 and A3/A5.

(2) Without workpiece identities, material-flow invariants cannot serve as reliable constraints. The benign violation rate q rises from 0.0054 on the training fold to 0.047 on the deployment stream, which caps single-message detection at only 0.21 for $\alpha = 0.01$. Approximate cross-case workpiece identities lower q to 2.3%, still above α, and end-to-end performance is the same as without the constraint, so the token constraint is not included in the detector. This limitation does not affect the three claims of this paper; it only means that the hard-constraint layer cannot yet cover cross-device material-flow forgery.

(3) The main evaluation uses a single public log, synthetic injections, and reimplemented baselines. The main evaluation has only one public log that provides both BPMN and the activity lifecycle; attacks are synthesized by a reproducible injector; the baselines are reimplemented from the structural constraints of the original papers rather than taken from official code; and no comparison is made with learned sequence models that use time features. The workflow splits within the log, the official error-containing version, and the unseen workflows serve only as stress tests and cannot substitute for a second external dataset; under a temporal split the conformal guarantee no longer holds automatically, and duration kernels cannot be transferred across scenarios directly (Section 6.6). This limitation threatens the external validity of the results on A2 and A3/A5, but does not affect Proposition 5.1.

7.3 Future Work

(1) Validate material-token constraints on production lines with workpiece identifiers. This addresses Limitation (2). With workpiece identities, if the benign violation rate in deployment can be brought down to the order of α, token conservation could complement the hard-constraint layer and detect cross-device material-flow forgery.

(2) Design a rule for allocating the false-alarm budget across channels without attack labels. This addresses Limitation (1). The current $\alpha/3$ is the even Bonferroni split under a parallel OR decision, and the disadvantage on A4/A6 may be related to the structural channel receiving only $\alpha/3$. An allocation rule must not use attack labels to tune thresholds in reverse.

(3) Add a second external dataset, official baseline implementations, and learned sequence baselines. This addresses Limitation (3). Testing the external validity of the conclusions requires another public log that provides both BPMN and the activity lifecycle, or official baseline implementations that accept this log format, complemented by learned sequence models with time features as competitors.

(4) Study online recalibration of the duration kernel. This addresses Limitation (3) and Section 6.6. The self-built scenarios show that transferring the duration kernel across scenarios sharply lowers A3 detection or raises false alarms, whereas the feasibility mask need not be re-estimated. Future work could update the duration kernel adaptively per product type under gate protection, so that the detector need not be refitted from scratch after a product changeover.

8 Data Availability

The Fischertechnik Trier smart-factory event log is released by its original authors [22]; a copy of the data is available on Figshare at `10.6084/m9.figshare.20130794`. The injection protocol, baseline reimplementations, diagnostic scripts,

and evaluation archives are in the accompanying code repository, available from the corresponding author during review and to be made public upon acceptance. The main tables and box plots are read from JSON archives, and no quantile is filled in by hand. Paper-level CSV files are aggregated from the archives by `py -m tools.export_experiments`. The self-built scenarios I/II/III (configurations A/B/C in the code) are regenerated from seeds by `algorithm/plant.py`. HAI is used only to check the data dictionary and is not imported into the CSV files.

Acknowledgments

We thank the University of Trier for releasing an IoT event log with a BPMN reference model, which made it possible for the feasibility mask to come from an external reference model rather than being learned from the log.

References

[1] Arya Adriansyah, Boudewijn F. van Dongen, and Wil M. P. van der Aalst. 2011. Conformance Checking Using Cost-Based Fitness Analysis. In *Proceedings of the 15th IEEE International Enterprise Distributed Object Computing Conference (EDOC)*. IEEE Computer Society, Helsinki, Finland, 55–64. doi:10.1109/EDOC.2011.12

[2] Anastasios N. Angelopoulos and Stephen Bates. 2023. Conformal Prediction: A Gentle Introduction. *Foundations and Trends in Machine Learning* 16, 4 (2023), 494–591. doi:10.1561/2200000101

[3] Rina Foygel Barber, Emmanuel J. Candès, Aaditya Ramdas, and Ryan J. Tibshirani. 2023. Conformal Prediction beyond Exchangeability. *The Annals of Statistics* 51, 2 (2023), 816–845. doi:10.1214/23-AOS2276

[4] Nisha Kumari Barsha and Neminath Hubballi. 2024. Anomaly Detection in SCADA Systems: A State Transition Modeling. *IEEE Transactions on Network and Service Management* 21, 3 (2024), 3511–3521. doi:10.1109/TNSM.2024.3373881

[5] Mauro Conti, Denis Donadel, and Federico Turrin. 2021. A Survey on Industrial Control System Testbeds and Datasets for Security Research. *IEEE Communications Surveys & Tutorials* 23, 4 (2021), 2248–2294. doi:10.1109/COMST.2021.3094360

[6] Martin Dobler, Michael Hellwig, Nuno Lopes, Ken Oakley, and Mike Winterburn. 2025. Systematic Review and Characterisation of Malicious Industrial Network Traffic Datasets. *International Journal of Information Security* 24, 5 (2025), 208. doi:10.1007/s10207-025-01126-9

[7] Danny Dolev and Andrew C. Yao. 1983. On the Security of Public Key Protocols. *IEEE Transactions on Information Theory* 29, 2 (1983), 198–208. doi:10.1109/TIT.1983.1056650

[8] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. 2017. DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, New York, NY, USA, 1285–1298. doi:10.1145/3133956.3134015

[9] Mahmoud Elsisi, Marnel Altius, Shun-Feng Su, and Chun-Lien Su. 2023. Robust Kalman Filter for Position Estimation of Automated Guided Vehicles Under Cyberattacks. *IEEE Transactions on Instrumentation and Measurement* 72 (2023), 1–12. doi:10.1109/TIM.2023.3250285

[10] Jonathan Goh, Sridhar Adepu, Khurum Nazir Junejo, and Aditya Mathur. 2017. A Dataset to Support Research in the Design of Secure Water Treatment Systems. In *Critical Information Infrastructures Security (CRITIS) (LNCS, Vol. 10242)*. Springer, Cham, 88–99. doi:10.1007/978-3-319-71368-7_8

[11] Niv Goldenberg and Avishai Wool. 2013. Accurate Modeling of Modbus/TCP for Intrusion Detection in SCADA Systems. *International Journal of Critical Infrastructure Protection* 6, 2 (2013), 63–75.

[12] Hakan Kayan, Matthew Nunes, Omer Rana, Pete Burnap, and Charith Perera. 2022. Cybersecurity of Industrial Cyber-Physical Systems: A Review. *Comput. Surveys* 54, 11s (2022), 229:1–229:35. doi:10.1145/3510410

[13] Ognjen Kundacina, Vladimir Vincan, Gorana Gojic, Vukan Ninkovic, and Dragisa Miskovic. 2025. Conformal Anomaly Detection for Predictive Maintenance in Thermal Power Plants. *IEEE Access* 13 (2025), 39738–39752. doi:10.1109/ACCESS.2025.3546451

[14] Tze Leung Lai. 1998. Information Bounds and Quick Detection of Parameter Changes in Stochastic Systems. *IEEE Transactions on Information Theory* 44, 7 (1998), 2917–2929.

[15] Max Landauer, Sebastian Onder, Florian Skopik, and Markus Wurzenberger. 2023. Deep Learning for Anomaly Detection in Log Data: A Survey. *Machine Learning with Applications* 12 (2023), 100470. doi:10.1016/j.mlwa.2023.100470

[16] Shang Li, Yasin Yılmaz, and Xiaodong Wang. 2015. Quickest Detection of False Data Injection Attack in Wide-Area Smart Grids. *IEEE Transactions on Smart Grid* 6, 6 (2015), 2725–2735.

[17] Chih-Yuan Lin, Simin Nadjm-Tehrani, and Mikael Asplund. 2018. Timing-Based Anomaly Detection in SCADA Networks. In *Critical Information Infrastructures Security (CRITIS) (LNCS, Vol. 10707)*. Springer, Cham, 48–59. doi:10.1007/978-3-319-99843-5_5

[18] Qin Lin, Sridhar Adepu, Sicco Verwer, and Aditya Mathur. 2018. TABOR: A Graphical Model-based Approach for Anomaly Detection in Industrial Control Systems. In *Proceedings of the 2018 ACM Asia Conference on Computer and Communications Security (AsiaCCS)*. ACM, New York, NY, USA, 525–536. doi:10.1145/3196494.3196546

[19] Hanxiao Liu, Yuqing Ni, Lihua Xie, and Karl Henrik Johansson. 2022. How Vulnerable is Innovation-Based Remote State Estimation: Fundamental Limits under Linear Attacks. *Automatica* 136 (2022), 110079. doi:10.1016/j.automatica.2021.110079

[20] Gary Lorden. 1971. Procedures for Reacting to a Change in Distribution. *The Annals of Mathematical Statistics* 42, 6 (1971), 1897–1908.

[21] Alexander Maier, Oliver Niggemann, Roman Just, Michael Jäger, and Asmir Vodenčarević. 2011. Anomaly Detection in Production Plants Using Timed Automata: Automated Learning of Models from Observations. In *Proceedings of the 8th International Conference on Informatics in Control, Automation and Robotics (ICINCO)*. SciTePress, Noordwijkerhout, The Netherlands, 363–369. doi:10.5220/0003538903630369

[22] Lukas Malburg, Joscha Grüger, and Ralph Bergmann. 2022. An IoT-Enriched Event Log for Process Mining in Smart Factories. arXiv preprint arXiv:2209.02702. doi:10.48550/arXiv.2209.02702 Dataset DOI: 10.6084/m9.figshare.20130794.

[23] Weibin Meng, Ying Liu, Yichen Zhu, Shenglin Zhang, Dan Pei, Yuqing Liu, Yihao Chen, Ruizhi Zhang, Shimin Tao, Pei Sun, and Rong Zhou. 2019. LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI)*. International Joint Conferences on Artificial Intelligence Organization, Macao, China, 4739–4745. doi:10.24963/ijcai.2019/658

[24] Yilin Mo and Bruno Sinopoli. 2009. Secure Control Against Replay Attacks. In *Proceedings of the 47th Annual Allerton Conference on Communication, Control, and Computing*. IEEE, Monticello, IL, USA, 911–918. doi:10.1109/ALLERTON.2009.5394956

[25] László Monostori, Botond Kádár, Thomas Bauernhansl, Shinsuke Kondoh, Soundar Kumara, Gunther Reinhart, Olaf Sauer, Günther Schuh, Wilfried Sihn, and Kanji Ueda. 2016. Cyber-Physical Systems in Manufacturing. *CIRP Annals* 65, 2 (2016), 621–641. doi:10.1016/j.cirp.2016.06.005

[26] Arunava Naha, André Teixeira, Anders Åhlén, and Subhrakanti Dey. 2023. Quickest Detection of Deception Attacks on Cyber–Physical Systems with a Parsimonious Watermarking Policy. *Automatica* 155 (2023), 111147. doi:10.1016/j.automatica.2023.111147

[27] Timo Nolle, Stefan Luettgen, Alexander Seeliger, and Max Mühlhäuser. 2018. Analyzing Business Process Anomalies Using Autoencoders. *Machine Learning* 107 (2018), 1875–1893. doi:10.1007/s10994-018-5702-8

[28] Timo Nolle, Stefan Luettgen, Alexander Seeliger, and Max Mühlhäuser. 2022. BINet: Multi-Perspective Business Process Anomaly Classification. *Information Systems* 103 (2022), 101458. doi:10.1016/j.is.2019.101458

[29] Neha Rino and Thomas Chatain. 2024. Timed Alignments with Mixed Moves. *Data & Knowledge Engineering* 154 (2024), 102366. doi:10.1016/j.datak.2024.102366

[30] Ronny Seiger, Lukas Malburg, Barbara Weber, and Ralph Bergmann. 2022. Integrating Process Management and Event Processing in Smart Factories: A Systems Architecture and Use Cases. *Journal of Manufacturing Systems* 63 (2022), 575–592. doi:10.1016/j.jmsy.2022.05.012

[31] Glenn Shafer and Vladimir Vovk. 2008. A Tutorial on Conformal Prediction. *Journal of Machine Learning Research* 9 (2008), 371–421.

[32] Hyeok-Ki Shin, Woomyo Lee, Jeong-Han Yun, and HyoungChun Kim. 2020. HAI 1.0: HIL-Based Augmented ICS Security Dataset. In *Proceedings of the 13th USENIX Workshop on Cyber Security Experimentation and Test (CSET)*. USENIX Association, Berkeley, CA, USA, 5 pages. https://www.usenix.org/conference/cset20/presentation/shin

[33] Xiaobin Tan and Hongsheng Xi. 2008. Hidden Semi-Markov Model for Anomaly Detection. *Appl. Math. Comput.* 205, 2 (2008), 562–567. doi:10.1016/j.amc.2008.05.028

[34] Wil M. P. van der Aalst, Arya Adriansyah, and Boudewijn F. van Dongen. 2012. Replaying History on Process Models for Conformance Checking and Performance Analysis. *WIREs Data Mining and Knowledge Discovery* 2, 2 (2012), 182–192.

[35] Denis Volkhonskiy, Evgeny Burnaev, Ilia Nouretdinov, Alexander Gammerman, and Vladimir Vovk. 2017. Inductive Conformal Martingales for Change-Point Detection. In *Proceedings of the Sixth Workshop on Conformal and Probabilistic Prediction and Applications (Proceedings of Machine Learning Research, Vol. 60)*. PMLR, Stockholm, Sweden, 132–153.

[36] Vladimir Vovk, Ilia Nouretdinov, and Alex Gammerman. 2003. Testing Exchangeability On-Line. In *Proceedings of the Twentieth International Conference on Machine Learning (ICML)*, Tom Fawcett and Nina Mishra (Eds.). AAAI Press, Menlo Park, CA, 768–775.

[37] Shuaiqi Yuan, Jipu Li, Chunjin Wang, and Xiaoge Zhang. 2026. Conformal Machine Learning for Reliable Anomaly Detection in Industrial Cyber–Physical Systems. *Reliability Engineering & System Safety* 274 (2026), 112417. doi:10.1016/j.ress.2026.112417